راهنمای کامل آمادگی ارائه

کاربرد هوش مصنوعی در پزشکی

پروژه‌ی PneumoniaCNN از دید طراحی زبان‌های برنامه‌نویسی

درس طراحی زبان‌های برنامه‌نویسی · حدود ۲۰ دقیقه · ۲۵ اسلاید اصلی + ۵ پیوست · زمان برنامه: ۱۹:۵۰ دقیقه

مسیر ارائه در برنامه: /presentation · کلید N برای یادداشت گوینده

فهرست

۱. نقشه‌ی ارائه و زمان‌بندی

۲. پیام اصلی و جمله‌های کلیدی

۳. اسلاید به اسلاید: متن کامل، نکات و پرسش‌های احتمالی

۴. پروژه‌ی PneumoniaCNN با جزئیات کامل

۵. مفاهیم طراحی زبان با جزئیات کامل

۶. بانک پرسش و پاسخ

۷. واژه‌نامه

۸. چک‌لیست روز ارائه و تمرین

۹. ادعاهای نیازمند راستی‌آزمایی

۱۰. منابع

> **چطور از این سند استفاده کنی؟** بخش ۱ و ۲ را حفظ کن تا ساختار را در ذهن داشته باشی. بخش ۳ را با صدای بلند و همراه با اسلایدها تمرین کن. بخش‌های ۴ و ۵ برای جواب دادن به سؤال‌های عمیق استاد است و لازم نیست همه‌اش را در ارائه بگویی. بخش ۶ را روز قبل از ارائه مرور کن.

۱. نقشه‌ی ارائه و زمان‌بندی

ارائه ۲۵ اسلاید اصلی و حدود ۱۹:۵۰ دقیقه است و چهار بخش دارد: زمینه‌ی پزشکی (کوتاه)، پروژه‌ی من، نگاه طراحی زبان (بخش اصلی برای این درس) و جمع‌بندی. پیوست‌ها فقط برای پاسخ به سؤال یا در صورت داشتن وقت اضافه است.

#	اسلاید	بخش	زمان	تجمعی
۱	عنوان و سؤال آغازین	زمینه‌ی پزشکی	۰:۳۰	۰:۳۰
۲	چرا پزشکی از AI استفاده می‌کند؟	زمینه‌ی پزشکی	۰:۴۰	۱:۱۰
۳	AI، ML و DL	زمینه‌ی پزشکی	۰:۳۰	۱:۴۰
۴	معماری سامانه	زمینه‌ی پزشکی	۰:۴۰	۲:۲۰
۵	شبکه‌ی عصبی به زبان ساده	زمینه‌ی پزشکی	۰:۴۵	۳:۰۵
۶	تصویربرداری، CNN و وظایف	زمینه‌ی پزشکی	۱:۰۰	۴:۰۵
۷	MASAI و غربالگری ماموگرافی	زمینه‌ی پزشکی	۰:۴۵	۴:۵۰
۸	IDx-DR	زمینه‌ی پزشکی	۰:۳۰	۵:۲۰
۹	کشف دارو، AlphaFold و TxGemma	زمینه‌ی پزشکی	۰:۴۵	۶:۰۵
۱۰	ابزارهای واقعی و دستیارهای سلامت	زمینه‌ی پزشکی	۰:۴۰	۶:۴۵
۱۱	معیارهای ارزیابی	زمینه‌ی پزشکی	۱:۰۰	۷:۴۵
۱۲	پروژه‌ی من: مسئله و داده	پروژه‌ی من	۰:۴۰	۸:۲۵
۱۳	پروژه‌ی من: مدل و تصمیم‌ها	پروژه‌ی من	۰:۴۵	۹:۱۰
۱۴	پروژه‌ی من: انتخاب آستانه	پروژه‌ی من	۰:۴۰	۹:۵۰
۱۵	پروژه‌ی من: نتایج	پروژه‌ی من	۱:۰۰	۱۰:۵۰
۱۶	پروژه‌ی من: سیستم و دمو	پروژه‌ی من	۰:۴۰	۱۱:۳۰
۱۷	پروژه‌ی من: محدودیت‌ها	پروژه‌ی من	۰:۳۵	۱۲:۰۵
۱۸	طراحی زبان: یک پروژه، چند زبان	طراحی زبان	۱:۰۰	۱۳:۰۵
۱۹	طراحی زبان: PyTorch به‌عنوان DSL	طراحی زبان	۱:۱۵	۱۴:۲۰
۲۰	طراحی زبان: سیستم نوع در مرز	طراحی زبان	۱:۱۵	۱۵:۳۵
۲۱	طراحی زبان: ویژگی‌ها در کد	طراحی زبان	۱:۱۵	۱۶:۵۰
۲۲	طراحی زبان: درس‌ها	طراحی زبان	۰:۴۵	۱۷:۳۵
۲۳	سوگیری، حریم خصوصی و مقررات	جمع‌بندی	۱:۰۰	۱۸:۳۵
۲۴	آیا AI جای پزشک را می‌گیرد؟	جمع‌بندی	۰:۳۰	۱۹:۰۵
۲۵	جمع‌بندی	جمع‌بندی	۰:۴۵	۱۹:۵۰

ایستگاه‌های زمانی: شروع پروژه (اسلاید ۱۲) حدود ۷:۴۵؛ شروع بخش طراحی زبان (اسلاید ۱۸) حدود ۱۲:۰۵؛ شروع چالش‌ها (اسلاید ۲۳) حدود ۱۷:۳۵. اگر عقب بودی، اسلایدهای ۷ تا ۱۰ (MASAI، IDx-DR، کشف دارو، نرم‌افزارها) را با یک جمله رد کن.

اسلایدهای پیوست

- پیوست الف: ادعاهایی که پیش از ارائه باید راستی‌آزمایی شوند
- پیوست ب: منابع و پیوندها
- پیوست ج: مینی‌پروژه‌ی تمرینی PyTorch (Wisconsin)
- پیوست د: پایش و پیش‌بینی خطر
- پیوست ه: درمان، جراحی و مدیریت اطلاعات

۲. پیام اصلی و جمله‌های کلیدی

پیام ارائه در سه جمله

1. هوش مصنوعی در پزشکی شواهد جدی دارد، اما هر عددی باید روی داده‌ی مستقل بازآزمایی شود.
2. در پروژه‌ی خودم، مدلی که روی Validation دقت ۹۶٫۴٪ داشت، روی Test مستقل به ۸۴٫۹٪ رسید و Specificity از ۹۳٪ به ۶۳٪ افتاد.
3. از دید طراحی زبان، این سیستم چندزبانه نشان داد PyTorch یک DSL درون‌زبانی است و مرز بین Python و TypeScript جایی است که نوع‌ها پاک می‌شوند و باید آگاهانه اعتبارسنجی شود.

جمله‌های کلیدی که باید دقیق بگویی

- «هشدار هوش مصنوعی یعنی دقیق‌تر نگاه کنیم، نه سرطان است.»
- «هر نرم‌افزار = یک کاربرد تخصصی + اعتبارسنجی علمی + مجوز.»
- «Accuracy به‌تنهایی کافی نیست؛ ۸۴٫۹٪ Accuracy پشتش ۸۶ هشدار کاذب از ۲۳۴ فرد سالم است.»
- «آستانه را روی Validation انتخاب کردم؛ Test فقط یک بار برای گزارش نهایی استفاده شد.»
- «مدل کوچک ساختم تا فرایند را بفهمم، نه تا تشخیص بدهم؛ و یاد گرفتم که عدد Validation، وعده‌ی دنیای واقعی نیست.»
- «PyTorch زبان جدیدی نیست؛ یک DSL درون‌زبانی است که از نحو خود Python استفاده می‌کند.»
- «نوع‌های TypeScript در زمان اجرا وجود ندارند؛ as یک ادعاست، نه یک بررسی.»
- «در نرم‌افزار پزشکی، هر خطایی که کامپایلر بگیرد، خطایی است که به بیمار نمی‌رسد.»
- «هوش مصنوعی جایگزین پزشک نمی‌شود، اما پزشکی که از آن استفاده کند، جایگزین پزشکی خواهد شد که استفاده نمی‌کند.»

۳. اسلاید به اسلاید

برای هر اسلاید: پیام اصلی، نکاتی که روی اسلاید است، متن کامل گوینده، هشدارهای راستی‌آزمایی و پرسش‌های احتمالی با پاسخ کوتاه.

بخش: زمینه‌ی پزشکی

اسلاید ۱: کاربرد هوش مصنوعی در پزشکی

زمان پیشنهادی: ۰:۳۰ دقیقه

> **پیام اصلی:** آیا پیش‌بینی دقیق برای استفاده‌ی پزشکی کافی است؟

روی اسلاید

- نیمه‌شب، بخش مراقبت‌های ویژه؛ نشانه‌هایی آن‌قدر ظریف که پزشک و پرستار خسته شاید نبینند.
- چه کسی مراقب است؟ بخشی از پاسخ امروز: الگوریتمی که هرگز نمی‌خوابد.

متن گوینده

با یک سؤال شروع می‌کنم. فرض کنید نیمه‌شب است و بیماری در بخش مراقبت‌های ویژه به‌آرامی رو به وخامت می‌رود. نشانه‌ها آن‌قدر ظریف‌اند که شاید پزشک و پرستار خسته هم متوجه نشوند. چه کسی مراقب است؟ امروز بخشی از پاسخ این است: الگوریتمی که هرگز نمی‌خوابد و خسته نمی‌شود.

در سال ۲۰۲۴ جایزه نوبل شیمی به دمیس هسابیس و جان جامپر برای سیستم هوش مصنوعی AlphaFold، و به دیوید بیکر برای طراحی پروتئین رسید. این اتفاق نشان داد هوش مصنوعی از یک وعده فناورانه به ابزاری با اعتبار علمی در بالاترین سطح تبدیل شده است.

در این ارائه ابتدا کوتاه توضیح می‌دهم چرا به هوش مصنوعی در پزشکی نیاز داریم و چند کاربرد واقعی را مرور می‌کنم. بعد پروژه‌ی خودم، PneumoniaCNN، را نشان می‌دهم و در بخش اصلی برای این درس، همین پروژه را از دید طراحی زبان‌های برنامه‌نویسی تحلیل می‌کنم: چند زبان، یک DSL درون‌زبانی، و سیستم نوع در مرز زبان‌ها. در پایان به چالش‌ها و این پرسش می‌رسم که آیا هوش مصنوعی جای پزشک را خواهد گرفت.

نیازمند راستی‌آزمایی

> ⚠ AlphaFold و نوبل ۲۰۲۴: نقش افراد را از منبع رسمی Nobel Prize بررسی کنید.

پرسش‌های احتمالی

س: این موضوع چه ربطی به درس طراحی زبان دارد؟

ج: پروژه یک سیستم چندزبانه است: TypeScript، Python، و C++ زیر PyTorch. در بخش آخر نشان می‌دهم PyTorch یک DSL درون‌زبانی است و سیستم نوع در مرز زبان‌ها چه نقشی دارد.

اسلاید ۲: چرا از هوش مصنوعی در پزشکی استفاده می‌کنیم؟

زمان پیشنهادی: ۰:۴۰ دقیقه

> **پیام اصلی:** هوش مصنوعی جایی کارآمد است که داده بسیار زیاد است ولی بهره‌برداری از آن کم.

روی اسلاید

- حجم داده‌ای که هیچ انسانی نمی‌تواند به‌طور کامل بررسی کند.

- سرعت و اولویت‌بندی در اورژانس و سکته مغزی.
- غربالگری و تشخیص زودهنگام.
- کاهش کار تکراری و دسترسی بیشتر به اطلاعات سلامت.

متن گوینده

اولین دلیل، حجم داده است. پزشکی با تصاویر، آزمایش‌ها، سیگنال‌های حیاتی، داده‌های ژنتیکی و یادداشت‌های بالینی روبه‌روست و هیچ انسانی نمی‌تواند همه این‌ها را به‌طور کامل بررسی کند. هوش مصنوعی دقیقاً در حوزه‌هایی کارآمد است که داده بسیار زیاد است ولی بهره‌برداری از آن کم.

دلیل دوم، سرعت است. در اورژانس و در بیماری‌هایی مثل سکته مغزی، هر دقیقه اهمیت دارد و اولویت‌بندی سریع تصاویر می‌تواند تفاوت ایجاد کند. دلیل سوم، تشخیص زودهنگام است: غربالگری و شناسایی الگوهایی که گاهی از چشم انسان پنهان می‌ماند. دلیل چهارم، کاهش کارهای تکراری متخصصان است، مثل خواندن هزاران ماموگرافی یا تهیه پیش‌نویس گزارش. دلیل پنجم، دسترسی بیشتر مردم به اطلاعات عمومی سلامت است.

تکمیلی: استفاده از AI باید با یک مسئله‌ی بالینی مشخص و داده‌ی باکیفیت شروع شود؛ «داشتن داده» به‌تنهایی دلیل کافی نیست.

پرسش‌های احتمالی

س: اگر داده زیاد است، چرا فقط آمار کلاسیک کافی نیست؟

ج: برای داده‌ی جدولی گاهی کافی است؛ اما برای تصویر خام، ویژگی‌ها باید از پیکسل‌ها یاد گرفته شوند و این کار شبکه‌های کانولوشنی است.

اسلاید ۳: هوش مصنوعی، یادگیری ماشین و یادگیری عمیق

زمان پیشنهادی: ۰:۳۰ دقیقه

> **پیام اصلی:** مدل باید متناسب با مسئله انتخاب شود؛ پیچیده‌تر بودن، تضمین عملکرد بهتر نیست.

روی اسلاید

- AI: عنوانی گسترده برای سامانه‌های هوشمند.
- ML: روش‌هایی که از داده الگو می‌آموزند.
- DL: شبکه‌های عصبی چندلایه برای نمایش‌های پیچیده.
- در پزشکی: کمک به تصمیم‌گیری، نه جایگزینی کامل پزشک.

متن گوینده

در پزشکی، هوش مصنوعی یعنی استفاده از الگوریتم‌ها و مدل‌های یادگیری ماشین برای تحلیل داده‌های پزشکی و کمک به تصمیم‌گیری. داده‌هایی که تحلیل می‌شوند شامل تصاویر پزشکی، نتایج آزمایشگاهی، سوابق درمانی، اطلاعات ژنتیکی و داده‌های سلامت افراد است. هدف، جایگزینی کامل پزشک نیست؛ افزایش سرعت و کمک به تصمیم‌گیری است.

هوش مصنوعی (AI) عنوانی گسترده است. یادگیری ماشین (ML) به روش‌هایی اشاره دارد که از داده الگو می‌آموزند. یادگیری عمیق (DL) از شبکه‌های عصبی چندلایه برای یادگیری نمایش‌های پیچیده استفاده می‌کند. انتخاب مدل باید تابع نوع ورودی، حجم داده، کیفیت برچسب، نیاز به تفسیرپذیری و محدودیت‌های اجرایی باشد؛ پیچیده‌تر بودن مدل به‌خودی‌خود تضمین‌کننده‌ی عملکرد بهتر نیست.

پرسش‌های احتمالی

س: فرق یادگیری ماشین با برنامه‌نویسی معمولی چیست؟

ج: در برنامه‌نویسی معمولی قاعده را ما می‌نویسیم؛ در یادگیری ماشین قاعده (وزن‌ها) از روی داده و برچسب استخراج می‌شود.

اسلاید ۴: از مسئله تا پایش: معماری یک سامانه‌ی پزشکی

زمان پیشنهادی: ۰۴۰ دقیقه

> **پیام اصلی:** «مدل دقیق» فقط یکی از اجزاست؛ داده، ارزیابی و پایش پس از استقرار هم تعیین‌کننده‌اند.

روی اسلاید

- Train برای یادگیری، Validation برای تنظیم، Test برای برآورد نهایی.
- External validation: مرکز، زمان یا جمعیت جدا از محل توسعه.
- Data leakage عملکرد را به‌شدت خوش‌بینانه نشان می‌دهد.

متن گوینده

برای اینکه یک سامانه‌ی هوش مصنوعی پزشکی را درست توصیف کنیم، باید پنج مرحله را جدا کنیم: تعریف کاربرد بالینی، گردآوری و برچسب‌گذاری داده، پیش‌پردازش، آموزش/اعتبارسنجی مدل، و ادغام کنترل‌شده در جریان کار درمان. «مدل دقیق» فقط یکی از اجزاست؛ کیفیت برچسب‌ها، نمایندگی جمعیت، خطاهای اندازه‌گیری، شیوه‌ی استفاده و پایش پس از استقرار نیز بر نتیجه اثر دارند.

Train: داده‌ای که مدل پارامترهایش را با آن یاد می‌گیرد. Validation: داده‌ای برای انتخاب معماری، تنظیم ابرپارامترها و تعیین آستانه. Test: داده‌ای که تا پایان توسعه کنار گذاشته می‌شود تا برآورد نهایی مستقل‌تر باشد. External validation: ارزیابی روی مرکز، بازه‌ی زمانی یا جمعیتی جدا از محل توسعه.

Data leakage: استفاده‌ی ناخواسته از اطلاعاتی که در زمان پیش‌بینی واقعی در دسترس نیستند یا انتقال اطلاعات بین بخش‌های داده. اگر نمونه‌های یک بیمار یا تصاویر نزدیک به هم در Train و Test پخش شوند، مدل ممکن است ویژگی‌های بیمار یا مرکز را حفظ کند؛ تقسیم داده باید با ساختار واقعی مسئله سازگار باشد.

پرسش‌های احتمالی

س: Data leakage در پروژه‌ی خودت ممکن است رخ داده باشد؟

ج: ممکن است: Validation از دل Train اصلی جدا شده و تضمینی نیست که تصاویر یک بیمار در هر دو نباشند. Test اما مجموعه‌ی جداست؛ برای همین Test را معیار اصلی گزارش کردم.

اسلاید ۵: شبکه‌ی عصبی به زبان ساده

زمان پیشنهادی: ۰۴۵ دقیقه

> **پیام اصلی:** پیش‌بینی ← محاسبه‌ی Loss ← گرادیان ← به‌روزرسانی وزن‌ها؛ برای چندین epoch.

روی اسلاید

- تابع زیان اختلاف پیش‌بینی با برچسب را می‌سنجد.
- پس‌انتشار خطا گرادیان را محاسبه می‌کند؛ بهینه‌ساز (مثلاً Adam) وزن‌ها را به‌روز می‌کند.
- کاهش Training Loss لزوماً به معنی تعمیم خوب نیست.
- Overfitting را با Validation Loss و داده‌ی مستقل پایش کنید.

متن گوینده

در یک شبکه‌ی عصبی پیش‌خور، ورودی از لایه‌ها عبور می‌کند و خروجی تولید می‌شود. تابع زیان اختلاف پیش‌بینی با برچسب را اندازه می‌گیرد. در پس‌انتشار خطا (Backpropagation)، گرادیان زیان نسبت به پارامترها محاسبه می‌شود و بهینه‌ساز، مثلاً Adam، وزن‌ها را به‌روزرسانی می‌کند. این چرخه برای چندین epoch تکرار می‌شود.

Epoch: یک بار عبور کامل از مجموعه‌ی آموزشی. Batch: بخشی از نمونه‌های آموزشی که در یک مرحله پردازش می‌شوند. Training loss: میزان خطا روی داده‌ی آموزش؛ کاهش آن لزوماً به معنای تعمیم خوب نیست. Overfitting: زمانی که مدل داده‌ی آموزش را بیش از حد یاد می‌گیرد و روی نمونه‌های جدید ضعیف‌تر عمل می‌کند.

پرسش‌های احتمالی

س: چرا Adam و نه SGD؟

ج: Adam نرخ یادگیری را برای هر پارامتر تطبیق می‌دهد و معمولاً بدون تنظیم زیاد سریع همگرا می‌شود؛ برای پروژه‌ی آموزشی انتخاب امن‌تری است.

س: Backpropagation چه ربطی به Autograd دارد؟

ج: Backpropagation همان مشتق‌گیری خودکار معکوس روی گراف شبکه است و Autograd پیاده‌سازی عمومی آن در PyTorch است.

اسلاید ۶: تشخیص از روی تصویر: CNN و سه وظیفه‌ی متفاوت

زمان پیشنهادی: ۱:۰۰ دقیقه

> **پیام اصلی:** هشدار هوش مصنوعی یعنی «دقیق‌تر نگاه کنیم»، نه «سرطان است».

روی اسلاید

- CT: خونریزی مغزی، ندول ریوی، اولویت‌بندی یافته‌های فوری.
- MRI: ساختارهای مغز، ضایعات مغزی، پروستات، مقایسه در طول زمان.
- سرطان: پستان، ریه، پروستات و سنجش حجم تومور مغزی.
- غربالگری ≠ تشخیص قطعی؛ ممکن است نمونه‌برداری لازم باشد.

متن گوینده

رایج‌ترین کاربردهای هوش مصنوعی در پزشکی امروز، پشتیبانی از تصمیم بالینی و تحلیل تصاویر پزشکی است. شبکه‌های عصبی عمیق روی هزاران تصویر سی‌تی‌اسکن، MRI، رادیوگرافی و ماموگرافی آموزش می‌بینند و الگوهایی را یاد می‌گیرند که گاهی از چشم انسان پنهان می‌ماند.

روند کار در بیمارستان این‌طور است: دستگاه تصویر را ثبت می‌کند، تصویر به سامانه ذخیره و مشاهده تصاویر یعنی PACS می‌رود، نرم‌افزار هوش مصنوعی آن را پردازش می‌کند و ممکن است ناحیه مشکوک را مشخص کند، اندازه‌گیری انجام دهد یا هشدار بدهد. بعد پزشک یافته‌ها را با علائم و سوابق بیمار بررسی می‌کند و گزارش نهایی با مسئولیت متخصص تهیه می‌شود.

در CT، سامانه‌ها می‌توانند نواحی مشکوک به خونریزی مغزی و ندول‌های ریوی را شناسایی کنند، بعضی ضایعات را اندازه بگیرند و تصاویر دارای یافته فوری را در صف جلو بیندازند. در MRI، کاربردها به ناحیه بدن بستگی دارد: بررسی ساختارهای مغز، ارزیابی ضایعات مغزی، تحلیل MRI پروستات و مقایسه تصاویر در طول زمان.

در تشخیص سرطان، چهار کاربرد مهم داریم: سرطان پستان در ماموگرافی، سرطان ریه از روی ندول‌های CT، سرطان پروستات از روی MRI، و تومورهای مغزی که هوش مصنوعی می‌تواند ناحیه و حجم آن‌ها را در طول درمان بسنجد. اما یک نکته حیاتی هست: غربالگری یعنی بررسی افراد برای یافتن نشانه‌های احتمالی، و شناسایی یک ناحیه مشکوک اثبات سرطان نیست.

CNN با فیلترهای کانولوشن الگوهای مکانی را استخراج می‌کند. لایه‌های ابتدایی ممکن است لبه‌ها و بافت‌های ساده را بیاموزند و لایه‌های عمیق‌تر ترکیب‌های پیچیده‌تری بسازند. این توصیف شهودی است؛ آنچه دقیقاً آموخته می‌شود به داده، معماری و روش آموزش وابسته است.

دموی زنده: در همین پروژه، مدل PneumoniaCNN یک نمونه‌ی ساده از طبقه‌بندی (Classification) روی رادیوگرافی قفسه سینه است.

پرسش‌های احتمالی

س: CNN چرا برای تصویر مناسب است؟

ج: دو فرض دارد: محلی بودن (پیکسل‌های نزدیک به هم مرتبط‌اند) و اشتراک وزن (یک فیلتر در همه جای تصویر استفاده می‌شود). این تعداد پارامتر را بسیار کم می‌کند.

س: MaxPool چه می‌کند؟

ج: در هر پنجره‌ی ۲×۲ بیشینه را نگه می‌دارد؛ ابعاد را نصف می‌کند و کمی مقاومت در برابر جابه‌جایی کوچک ایجاد می‌کند. ۲۸ ← ۱۴ ← ۷.

اسلاید ۷: شواهد در ماموگرافی: MASAI

زمان پیشنهادی: ۰:۴۵ دقیقه

> **پیام اصلی:** شواهد را در چارچوب طراحی مطالعه بخوانیم: جمعیت، مقایسه‌گر، پیامد و عدد دقیق.

روی اسلاید

- مطالعه‌ی بزرگ سوئدی MASAI در Lancet Oncology: کشف سرطان بیشتر و کاهش بار کاری رادیولوژیست.
- پژوهش‌های چندمرکزی در کره جنوبی: افزایش شناسایی سرطان پستان.
- نتایج گزارش‌شده در سکته و صرع را «گزارش» بدانیم، نه حکم قطعی.

متن گوینده

قوی‌ترین شواهد در ماموگرافی است. مطالعه بزرگ سوئدی MASAI که در Lancet Oncology منتشر شد نشان داد غربالگری با کمک هوش مصنوعی سرطان‌های بیشتری پیدا می‌کند و بار کاری رادیولوژیست‌ها را به‌شدت کاهش می‌دهد. در کره جنوبی هم پژوهش‌های چندمرکزی نشان داده‌اند این فناوری می‌تواند شناسایی سرطان پستان را بیشتر کند.

در منابع آموزشی گزارش شده که نرم‌افزاری در بررسی تصاویر مغزی بیماران سکته تا دو برابر دقیق‌تر از متخصصان بوده و در مطالعه‌ای درباره صرع، ابزاری حدود ۶۴ درصد ضایعات مغزی را شناسایی کرده که رادیولوژیست‌ها ندیده بودند. این‌ها را نتایج گزارش‌شده می‌دانم، نه حکم قطعی، چون جزئیات مطالعه اصلی را باید جداگانه بررسی کرد.

نیازمند راستی‌آزمایی

> ⚠️ MASAI: مقاله‌ی اصلی، نوع پیامد، جمعیت، مقایسه و عدد دقیق از ناشر بررسی شود.
>
> ⚠️ «دو برابر دقیق‌تر» در سکته: منبع اصلی و تعریف دقت مشخص شود، وگرنه حذف یا محتاطانه بیان شود.

⚠️ ۶۴٪ ضایعات صرع: مقاله‌ی اصلی و مخرج عدد بررسی شود.

پرسش‌های احتمالی

س: عدد دقیق MASAI چقدر است؟

ج: عدد دقیق را از مقاله‌ی اصلی Lancet Oncology نقل نکرده‌ام و در فهرست راستی‌آزمایی گذاشته‌ام؛ ادعای کیفی این است که کشف سرطان افزایش و بار کاری کاهش یافته.

اسلاید ۸: IDx-DR: یک محصول، یک کاربرد، یک مجوز

زمان پیشنهادی: ۰:۳۰ دقیقه

پیام اصلی: محصول مشخص، کاربرد مشخص و مجوز مشخص؛ نه «هوش مصنوعی همه‌کاره».

روی اسلاید

- ۲۰۱۸: اولین سیستم تشخیصی خودمختاری که FDA تأیید کرد.
- تشخیص رتینوپاتی دیابتی از روی تصویر شبکیه.
- بدون اینکه پزشک تصویر را تفسیر کند.

متن گوینده

یک مثال تاریخی هم دارم: سیستم IDx-DR در ۲۰۱۸ اولین سیستم تشخیصی خودمختاری بود که FDA تأیید کرد. این سیستم رتینوپاتی دیابتی را از روی تصویر شبکیه تشخیص می‌دهد، بدون اینکه پزشک تصویر را تفسیر کند.

تکمیلی: وضعیت مقرراتی هر محصول باید برای کاربرد، نسخه، کشور و تاریخ موردنظر بررسی شود؛ approval، clearance و نشان CE اصطلاحات یکسانی نیستند.

نیازمند راستی‌آزمایی

⚠️ صفحه‌ی رسمی FDA و اصطلاح دقیق تنظیم‌گری برای IDx-DR بررسی شود.

پرسش‌های احتمالی

س: خودمختار یعنی بدون پزشک؟

ج: یعنی تفسیر تصویر توسط پزشک لازم نیست و خروجی مستقیم صادر می‌شود؛ اما در چارچوب کاربرد مشخص و مجوز همان کاربرد.

اسلاید ۹: کشف دارو و زیست‌شناسی محاسباتی

زمان پیشنهادی: ۰:۴۵ دقیقه

پیام اصلی: پیش‌بینی محاسباتی جای آزمایش زیستی و کارآزمایی بالینی را نمی‌گیرد.

روی اسلاید

- MIT: کشف آنتی‌بیوتیک‌های halicin و abaucin در برابر باکتری‌های مقاوم.
- AlphaFold: پیش‌بینی ساختار سه‌بعدی پروتئین از توالی؛ نوبل ۲۰۲۴.
- TxGemma (Google، بر پایه‌ی Gemma 2): پیش‌بینی سمیت و ویژگی‌های مولکولی، مثلاً از SMILES.
- TxGemma ابزار پژوهش دارویی است، نه ابزار تشخیص بالینی.

متن گوینده

کشف دارو معمولاً طولانی‌ترین و پرهزینه‌ترین بخش توسعه دارو است. هوش مصنوعی از دو راه کمک می‌کند: طراحی بهتر ترکیبات و غربال سریع میلیون‌ها مولکول. مدل‌هایی مانند TxGemma نیز برای پیش‌بینی برخی ویژگی‌های مولکولی و اولویت‌بندی ترکیبات در پژوهش دارویی ساخته شده‌اند؛ با این حال خروجی محاسباتی آن‌ها جای آزمایش‌های زیستی و کارآزمایی بالینی را نمی‌گیرد.

گروه MIT با هوش مصنوعی آنتی‌بیوتیک halicin و بعد abaucin را کشف کرد؛ داروهایی در برابر باکتری‌های مقاوم، در دوره‌ای که مقاومت آنتی‌بیوتیکی بحران جهانی است. AlphaFold ساختار سه‌بعدی پروتئین را از روی توالی آمینواسیدی پیش‌بینی می‌کند، که ده‌ها سال از بزرگ‌ترین چالش‌های زیست‌شناسی بود و نوبل ۲۰۲۴ را رقم زد.

TxGemma خانواده‌ای از مدل‌های با وزن‌های در دسترس از Google است که بر پایه‌ی Gemma 2 ساخته و برای وظایف پژوهش و توسعه‌ی درمان تنظیم شده‌اند: پیش‌بینی سمیت و ویژگی‌های مولکول (مثلاً عبور از سد خونی‌مغزی)، تخمین ویژگی‌هایی مانند لیپوفیلیسیته، حساسیت رده‌ی سلولی و affinity binding، گفت‌وگو درباره‌ی مسئله‌ی پژوهشی با نسخه‌های Chat، و ترکیب در گردش‌کار چندمرحله‌ای. Agentic-Tx یک سامانه‌ی عامل‌محور جداگانه است و نباید آن را با خود مدل TxGemma یکی دانست.

نسخه‌ها: Predict در اندازه‌های 2B، 9B و 27B؛ Chat در اندازه‌های 9B و 27B. نسخه‌ی Chat لزوماً در پیش‌بینی خام از Predict بهتر نیست. عملکرد باید برای همان وظیفه، نوع مولکول و جمعیت داده سنجیده شود.

نیازمند راستی‌آزمایی

⚠️ Halicin، abaucin و TxGemma: مقاله یا مستندات اصلی هر کدام جداگانه ذکر شود.

پرسش‌های احتمالی

س: TxGemma دارو می‌سازد؟

ج: نه. ویژگی‌های مولکول مثل سمیت را پیش‌بینی و ترکیبات را اولویت‌بندی می‌کند. آزمایش آزمایشگاهی و کارآزمایی بالینی همچنان لازم است.

اسلاید ۱۰: نرم‌افزارهای واقعی و دستیارهای سلامت

زمان پیشنهادی: ۰:۴۰ دقیقه

پیام اصلی: هر نرم‌افزار = یک کاربرد تخصصی + اعتبارسنجی علمی + مجوز و اتصال به سیستم بیمارستان.

روی اسلاید

- FDA ابزارهای AI را به‌عنوان تجهیز پزشکی نرم‌افزاری بررسی می‌کند؛ بیشترشان در رادیولوژی‌اند.
- استفاده در چند مرکز به معنی استفاده‌ی سراسری نیست.
- دستیار متنی ≠ ابزار اعتبارسنجی‌شده برای تحلیل MRI یا تشخیص سرطان.

متن گوینده

تا اینجا از کاربردها گفتم. حالا چند نرم‌افزار واقعی را معرفی می‌کنم. نکته مهم این است که هر نرم‌افزار برای یک کاربرد تخصصی طراحی و اعتبارسنجی شده و برای همه انواع تصاویر مناسب نیست.

این ابزارها عمدتاً برای مراکز درمانی و متخصصان طراحی شده‌اند و ممکن است برای استفاده به مجوز، حساب سازمانی یا اتصال به سیستم‌های بیمارستانی نیاز داشته باشند. از نظر تنظیم‌گری، FDA آمریکا ابزارهای هوش مصنوعی را به‌عنوان تجهیز پزشکی نرم‌افزاری بررسی می‌کند و تعداد ابزارهای تأییدشده از هزار مورد گذشته است که بیشترشان در رادیولوژی‌اند. در بریتانیا هم برخی مراکز از این نرم‌افزارها استفاده می‌کنند، اما استفاده در چند مرکز به معنی استفاده سراسری نیست.

همه سامانه‌ها مخصوص بیمارستان نیستند. Ada Health علائم کاربر را می‌گیرد، سؤال‌های تکمیلی می‌پرسد و علت‌های احتمالی و اقدام بعدی را پیشنهاد می‌دهد؛ اما برای تشخیص قطعی طراحی نشده است. ChatGPT Health به درک اطلاعات سلامت، توضیح نتایج آزمایش و آماده‌شدن برای ویزیت کمک می‌کند و دسترسی به آن بسته به کشور و حساب کاربری متفاوت است.

مدل‌های زبانی بزرگ مثل ChatGPT و Med-PaLM 2 گوگل وارد گفت‌وگوی پزشکی شده‌اند. در برخی مطالعات پزشکان بخش بزرگی از پاسخ‌هایشان را تأیید کرده‌اند، اما دقتشان با ورودی ناقص افت می‌کند و دچار «هالوسینیشن» می‌شوند، یعنی با اطمینان کامل اطلاعات یا منابع ساختگی می‌دهند. در پزشکی این خطا می‌تواند خطرناک باشد.

پس تفاوت را به خاطر بسپارید: دستیار متنی می‌تواند درباره علائم و آزمایش گفت‌وگو کند، اما این به معنی اعتبارسنجی برای تحلیل MRI یا تشخیص سرطان نیست.

نیازمند راستی‌آزمایی

⚠️ «بیش از هزار ابزار AI مورد تأیید FDA»: تاریخ و تعریف فهرست از صفحه‌ی رسمی FDA بررسی شود.

⚠️ ChatGPT Health: وضعیت فعلی و دسترسی جغرافیایی در زمان ارائه بررسی شود.

پرسش‌های احتمالی

س: چرا ChatGPT را ابزار تشخیص حساب نمی‌کنید؟

ج: برای تحلیل تصویر پزشکی اعتبارسنجی و مجوز ندارد و ممکن است با اطمینان کامل پاسخ ساختگی بدهد (hallucination).

اسلاید ۱۱: معیارهای ارزیابی: Accuracy به‌تنهایی کافی نیست

زمان پیشنهادی: ۱:۰۰ دقیقه

پیام اصلی: انتخاب آستانه باید بر اساس کاربرد، پیامدها و ظرفیت نظام سلامت باشد، نه صرفاً ۰٫۵.

روی اسلاید

- در غربالگری، هزینه‌ی منفی کاذب (FN) ممکن است بالا باشد.
- مثبت کاذب (FP) اضطراب، آزمایش تکمیلی و هزینه ایجاد می‌کند.
- ROC-AUC بالا کالیبراسیون یا سود بالینی را اثبات نمی‌کند.

متن گوینده

TP یعنی مثبت درست، FN منفی کاذب، TN منفی درست و FP مثبت کاذب. در کاربرد غربالگری، هزینه‌ی منفی کاذب ممکن است بالا باشد؛ در عین حال، مثبت کاذب می‌تواند اضطراب، آزمایش تکمیلی و هزینه ایجاد کند. انتخاب آستانه باید بر اساس کاربرد، پیامدها و ظرفیت نظام سلامت انجام شود، نه صرفاً با انتخاب خودکار ۰٫۵.

ROC-AUC بالا نمی‌گوید که احتمال‌های مدل کالیبره‌اند، در مرکز دیگر نیز همان عملکرد را دارند یا استفاده از مدل پیامد بیمار را بهتر می‌کند. این‌ها پرسش‌های جداگانه‌اند. Calibration: آیا پیش‌بینی ۰٫۸ تقریباً در ۸۰٪ موارد درست رخ می‌دهد؟

پرسش‌های احتمالی

س: چرا F1 کلاس پنومونی ۰٫۸۹ است ولی Accuracy ۰٫۸۵؟

ج: F1 فقط کلاس پنومونی را می‌سنجد که مدل در آن خوب است؛ Accuracy خطای زیاد کلاس نرمال (۸۶ مثبت کاذب) را هم شامل می‌شود.

س: Macro avg و Weighted avg فرقشان چیست؟

ج: Macro میانگین ساده‌ی دو کلاس است (۰٫۸۲۵ برای F1)؛ Weighted بر اساس تعداد نمونه‌ی هر کلاس وزن می‌دهد (۰٫۸۴۱). در داده‌ی نامتوازن Macro صادق‌تر است.

بخش: پروژه‌ی من

اسلاید ۱۲: پروژه‌ی من: PneumoniaCNN — مسئله و داده

زمان پیشنهادی: ۰:۴۰ دقیقه

> **پیام اصلی:** طبقه‌بندی دودویی رادیوگرافی قفسه سینه: نرمال یا پنومونی؛ ابزار آموزشی، نه تشخیص.

روی اسلاید

- داده: PneumoniaMNIST از مجموعه‌ی MedMNIST v2؛ رادیوگرافی قفسه سینه‌ی کودکان.
- ۵۸۵۶ تصویر خاکستری ۲۸×۲۸ در سه بخش Train، Validation و Test.
- عدم توازن: تصویر پنومونی حدود ۲٫۹ برابر نرمال.
- Test مجموعه‌ای جداست؛ Validation از دل Train برداشته شده.

متن گوینده

حالا از کلیات به کار خودم می‌رسم. مسئله‌ی من یک طبقه‌بندی دودویی است: آیا رادیوگرافی قفسه سینه الگوی پنومونی دارد یا نرمال است؟ همان وظیفه‌ی Classification که در اسلاید تصویربرداری گفتم.

داده PneumoniaMNIST است، بخشی از بنچمارک MedMNIST v2 که از دیتاست رادیوگرافی کودکان Kermany و همکاران ساخته شده. تصاویر به ۱×۲۸×۲۸ کوچک شده‌اند. ۴۷۰۸ تصویر برای آموزش، ۵۲۴ برای Validation و ۶۲۴ برای Test. Validation از تقسیم ۹ به ۱ Train اصلی ساخته شده، ولی Test همان مجموعه‌ی جداگانه‌ی دیتاست اصلی است؛ این تفاوت بعداً در نتایج خودش را نشان می‌دهد.

داده نامتوازن است: در Validation ۱۳۵ نرمال در برابر ۳۸۹ پنومونی و در Test ۲۳۴ در برابر ۳۹۰. به همین دلیل Accuracy به‌تنهایی معیار مناسبی نیست.

پرسش‌های احتمالی

س: چرا ۲۸×۲۸؟ این خیلی کم است.

ج: PneumoniaMNIST استاندارد بنچمارک MedMNIST است؛ آموزش روی CPU سریع است و هدف یادگیری فرایند بود. محدودیتش را هم صریح در اسلاید محدودیت‌ها گفته‌ام.

س: داده از کجاست؟

ج: MedMNIST v2، برگرفته از دیتاست رادیوگرافی قفسه سینه‌ی کودکان Kermany و همکاران؛ Train و Validation به نسبت ۹ به ۱ از Train اصلی و Test همان مجموعه‌ی جداگانه‌ی اصلی.

اسلاید ۱۳: مدل و تصمیم‌های مهندسی

زمان پیشنهادی: ۰:۴۵ دقیقه

> **پیام اصلی:** یک CNN کوچک با حدود ۱۰۵ هزار پارامتر؛ ساخته‌شده برای فهم فرایند، نه رقابت با مدل‌های بزرگ.

روی اسلاید

- دو لایه‌ی کانولوشن (۱۶ و ۳۲ فیلتر) و دو لایه‌ی Linear.
- Loss وزن‌دار برای جبران عدم توازن کلاس‌ها.
- آستانه‌ی ۰٫۳ روی Validation انتخاب شد، نه با دست‌کاری Test.
- اولویت طراحی: بیمار از دست نرود (Sensitivity بالا).

متن گوینده

معماری عمداً ساده است: ورودی ۲۸×۲۸ خاکستری، دو بلوک Conv و ReLU و MaxPool با ۱۶ و ۳۲ فیلتر، سپس یک لایه‌ی Linear با ۶۴ نورون و خروجی دوکلاسه با Softmax. کل مدل ۱۰۵٬۳۴۶ پارامتر دارد؛ حدود یک‌صدم ResNet-18.

سه تصمیم مهندسی گرفتم. اول، چون تصاویر پنومونی تقریباً سه برابر نرمال‌اند، Loss را وزن‌دار کردم تا مدل کلاس کم‌تعداد را نادیده نگیرد. دوم، آستانه را روی Validation انتخاب کردم و Test را فقط یک بار برای گزارش نهایی استفاده کردم. سوم، آستانه را ۰٫۳ گذاشتم، نه ۰٫۵؛ چون در غربالگری از دست دادن بیمار هزینه‌ی بیشتری از یک هشدار کاذب دارد.

پرسش‌های احتمالی

س: عدد ۱۵۶۸ از کجا آمده؟

ج: ۳۲ کانال × ۷ × ۷. دو MaxPool ابعاد ۲۸ را به ۱۴ و بعد ۷ می‌رسانند و padding=1 در کانولوشن ابعاد را ثابت نگه می‌دارد.

س: تعداد پارامترها را چطور حساب کردی؟

ج: Conv1: ۱۶×۹+۱۶=۱۶۰؛ Conv2: ۳۲×۱۶×۹+۳۲=۴۶۴۰؛ fc1: ۱۵۶۸×۶۴+۶۴=۱۰۰۴۱۶؛ fc2: ۶۴×۲+۲=۱۳۰. جمع: ۱۰۵٬۳۴۶.

س: Loss وزن‌دار دقیقاً چه کرد؟

ج: به خطای کلاس کم‌تعداد (نرمال) وزن بیشتری داد تا مدل فقط با گفتن «پنومونی» به دقت بالا نرسد.

اسلاید ۱۴: انتخاب آستانه روی Validation

زمان پیشنهادی: ۰:۴۰ دقیقه

> **پیام اصلی:** آستانه‌ی پایین‌تر بیمار کمتری را از دست می‌دهد، اما هشدار کاذب بیشتری می‌سازد.

روی اسلاید

- آستانه‌ی ۰٫۳: فقط ۱۰ بیمار از ۳۸۹ جا افتاد (FN=10).
- آستانه‌ی ۰٫۷: FN به ۴۴ رسید؛ بیش از ۴ برابر.
- انتخاب ۰٫۳ یعنی ترجیح حساسیت برای کاربرد غربالگری.

متن گوینده

این جدول همان آزمایشی است که در اسلاید معیارها گفتم، این بار روی مدل خودم. روی ۵۲۴ تصویر Validation، آستانه را از ۰٫۳ تا ۰٫۷ تغییر دادم. با بالا رفتن آستانه، Specificity از ۹۳٪ به ۹۹٪ می‌رسد، اما Sensitivity از ۹۷٪ به ۸۹٪ می‌افتد و تعداد بیماران جاافتاده از ۱۰ به ۴۴ می‌رسد.

برای ابزاری که نقش غربالگری دارد، ۰٫۳ را انتخاب کردم. نکته‌ی روش‌شناختی مهم: این انتخاب فقط با Validation انجام شد.

پرسش‌های احتمالی

س: چرا آستانه را روی Test تنظیم نکردی؟

ج: چون آن‌وقت Test دیگر داده‌ی دیده‌نشده نبود و گزارش خوش‌بینانه می‌شد؛ Test فقط یک بار برای گزارش نهایی استفاده شد.

س: آستانه‌ی بهتر چیست؟

ج: به هزینه‌ی خطاها بستگی دارد. با نتایج Test، آستانه‌ی بالاتر احتمالاً تعادل بهتری می‌دهد؛ روش اصولی انتخاب با معیاری مثل Youden روی Validation مستقل از بیمار است.

اسلاید ۱۵: نتایج: Validation در برابر Test

زمان پیشنهادی: ۱:۰۰ دقیقه

پیام اصلی: Accuracy از ۹۶٫۴٪ به ۸۴٫۹٪ رسید؛ مدل بیمار را پیدا می‌کند، اما سالم بودن را خوب تشخیص نمی‌دهد.

روی اسلاید

- Sensitivity پایدار ماند: ۹۷٫۴٪ ← ۹۷٫۹٪؛ فقط ۸ بیمار از ۳۹۰ جا افتاد.
- Specificity افت کرد: ۹۳٫۳٪ ← ۶۳٫۲٪؛ ۸۶ نفر از ۲۳۴ فرد سالم هشدار گرفتند.
- Accuracy ۸۴٫۹٪ به‌تنهایی این ضعف را پنهان می‌کند.
- این شکاف همان Distribution Shift است که پیش‌تر گفتیم.

متن گوینده

مهم‌ترین اسلاید پروژه این است. روی Validation و با آستانه‌ی ۰٫۳، Accuracy ۹۶٫۴٪ بود. روی Test که داده‌ای جدا و دیده‌نشده است، Accuracy به ۸۴٫۹٪ رسید.

ولی عدد کلی داستان را کامل نمی‌گوید. Sensitivity تقریباً ثابت ماند، ۹۷٫۹٪؛ یعنی از ۳۹۰ بیمار فقط ۸ نفر جا افتادند. در عوض Specificity از ۹۳٪ به ۶۳٪ افتاد: ۸۶ نفر از ۲۳۴ کودک سالم برچسب پنومونی گرفتند. Precision پنومونی ۸۱٫۶٪ و F1 آن ۸۹٪ است.

چرا این شکاف؟ Validation از دل همان Train اصلی آمده و شبیه آن است، اما Test مجموعه‌ای جداست. این دقیقاً همان Distribution Shift و فاصله‌ی آزمایشگاه تا بالین است که درباره‌ی مدل سپسیس Epic گفتم؛ این بار در مقیاس کوچک، در پروژه‌ی خودم دیدمش.

برای مقایسه: در مقاله‌ی MedMNIST v2، ResNet-18 روی همین داده و با همین ورودی ۲۸×۲۸ به ACC=0.854 رسیده است. مدل من با حدود یک درصد پارامترها به 0.849 رسید؛ پس مدل کوچک ضعیف نبود، محدودیت اصلی داده و وضوح است.

پرسش‌های احتمالی

س: چرا Specificity روی Test این‌قدر افت کرد؟

ج: Validation از همان توزیع Train است ولی Test جداگانه جمع شده (Distribution Shift)؛ آستانه‌ی ۰٫۳ هم مدل را به سمت پنومونی متمایل کرده. نتیجه: ۸۶ مثبت کاذب از ۲۳۴ نرمال.

س: ماتریس Validation در گزارش تو FN=23 دارد؛ چرا اینجا ۱۰ است؟

ج: آن ماتریس با آستانه‌ی ۰٫۵ بود. برای مقایسه‌ی منصفانه هر دو را با ۰٫۳ آورده‌ام؛ عددهای ۰٫۳ از جدول جاروب آستانه آمده‌اند.

س: با ResNet مقایسه کردی؟

ج: از گزارش منتشرشده‌ی ResNet-18 :v2 MedMNIST با ورودی ۲۸ روی همین Test به ACC=0.854 رسیده؛ مدل من 0.849 با حدود یک درصد پارامتر.

اسلاید ۱۶: از مدل تا محصول: معماری نرم‌افزار و دموی زنده

زمان پیشنهادی: ۰:۴۰ دقیقه

> **پیام اصلی:** مرورگر ← PyTorch ← FastAPI ← Next.js؛ تصویر هیچ‌جا ذخیره نمی‌شود.

روی اسلاید

- فرانت‌اند: Next.js و TypeScript؛ احتمال هر کلاس و آستانه را نشان می‌دهد.
- بک‌اند: FastAPI؛ کنترل فرمت و حجم و پیش‌پردازش.
- حریم خصوصی از طراحی: تصویر فقط در حافظه پردازش می‌شود.
- نمونه‌ی دمو: پنومونی با احتمال ۹۹٫۶۸٪.

متن گوینده

مدل به‌تنهایی محصول نیست. یک رابط وب با Next.js ساختم که تصویر را می‌گیرد و به یک API با FastAPI می‌فرستد. API فرمت و حجم فایل را کنترل می‌کند، تصویر را به ۲۸×۲۸ خاکستری تبدیل می‌کند و مدل PyTorch پیش‌بینی را برمی‌گرداند. تصویر ذخیره نمی‌شود.

در رابط، فقط برچسب نشان داده نمی‌شود؛ احتمال هر کلاس، آستانه‌ی تصمیم و هشدار «نزدیک به آستانه» هم نمایش داده می‌شود تا کاربر بفهمد مدل چقدر مطمئن است.

(دموی زنده: یک تصویر نرمال و یک تصویر پنومونی از نمونه‌ها را تحلیل کنید.) در اجرای نهایی، یک تصویر پنومونی با احتمال ۹۹٫۶۸٪ تشخیص داده شد. اما یادمان باشد: احتمال‌ها کالیبره نشده‌اند؛ ۹۹٪ یعنی مدل مطمئن است، نه اینکه حتماً درست می‌گوید.

پرسش‌های احتمالی

س: پیش‌پردازش سرور با آموزش یکی است؟

ج: سرور تصویر را خاکستری و ۲۸×۲۸ می‌کند و به بازه‌ی ۰ تا ۱ می‌برد. PneumoniaMNIST تصویر را اول از وسط مربعی برش داده؛ سرور فقط تغییر اندازه می‌دهد. این یک ناهماهنگی کوچک آموزش و سرویس (training–serving skew) است و در کار بعدی باید یکسان شود.

س: ۹۹٫۶۸٪ یعنی مدل درست می‌گوید؟

ج: نه؛ یعنی مدل مطمئن است. احتمال‌ها کالیبره نشده‌اند و ۸۶ مثبت کاذب روی Test نشان می‌دهد اطمینان بالا لزوماً درستی نیست.

اسلاید ۱۷: محدودیت‌ها و قدم بعدی

زمان پیشنهادی: ۰:۳۵ دقیقه

> **پیام اصلی:** عدد خوب روی Validation، تضمین عملکرد در دنیای واقعی نیست.

متن گوینده

محدودیت‌ها را صریح می‌گویم. داده از کودکان است و برای بزرگسالان سنجیده نشده. وضوح ۲۸×۲۸ بسیار پایین است. Specificity روی Test فقط ۶۳٪ است. تقسیم Train و Validation در سطح بیمار تضمین نشده و ممکن است تصاویر یک بیمار در هر دو بخش باشد. احتمال‌ها هم کالیبره نشده‌اند.

قدم‌های بعدی: استفاده از وضوح ۲۲۴×۲۲۴ و Data Augmentation، Transfer Learning، تقسیم داده بر اساس بیمار و اعتبارسنجی روی یک دیتاست بیرونی، Grad-CAM برای اینکه ببینیم مدل به کدام ناحیه نگاه می‌کند، و کالیبراسیون احتمال‌ها.

پرسش‌های احتمالی

س: اگر وقت بیشتری داشتی اول چه می‌کردی؟

ج: تقسیم داده بر اساس بیمار، ورودی با وضوح بالاتر و Transfer Learning، و بعد Grad-CAM برای اینکه ببینم مدل واقعاً به ریه نگاه می‌کند.

بخش: طراحی زبان

اسلاید ۱۸: از دید طراحی زبان: یک پروژه، چند زبان و پارادایم

زمان پیشنهادی: ۱:۰۰ دقیقه

> **پیام اصلی:** هر لایه زبانی را گرفته که با مسئله‌اش جور است؛ و مرز بین زبان‌ها همان‌جایی است که خطا متولد می‌شود.

روی اسلاید

- Python: نوع‌دهی پویا ولی قوی؛ کامپایل به bytecode و اجرا روی ماشین مجازی CPython.
- TypeScript: نوع‌دهی ایستا، ساختاری و تدریجی؛ نوع‌ها پیش از اجرا پاک می‌شوند.
- C++/CUDA زیر PyTorch: محاسبه‌ی سنگین؛ Python فقط هماهنگ‌کننده است.
- JSON قرارداد مشترک دو زبان؛ JSX و CSS زبان‌های اعلانی.

متن گوینده

حالا همین پروژه را از دید درس طراحی زبان‌های برنامه‌نویسی نگاه می‌کنم. این سیستم کوچک دست‌کم پنج زبان دارد و هر کدام برای کاری انتخاب شده که در آن قوی است.

فرانت‌اند با TypeScript و TSX نوشته شده: سیستم نوع ایستا، ساختاری (structural) و تدریجی (gradual). ساختاری یعنی دو نوع اگر شکلشان یکی باشد سازگارند، نه اینکه اسمشان یکی باشد. TypeScript به JavaScript ترجمه (transpile) می‌شود و در این مرحله همه‌ی نوع‌ها پاک می‌شوند؛ به این می‌گویند Type Erasure. پس در زمان اجرا هیچ نوعی وجود ندارد.

بک‌اند با Python است: نوع‌دهی پویا (dynamic) ولی قوی (strong)؛ یعنی نوع متغیر در زمان اجرا مشخص می‌شود، اما Python مثلاً عدد و رشته را بی‌صدا با هم جمع نمی‌کند. Python ابتدا به bytecode کامپایل و سپس روی ماشین مجازی CPython تفسیر می‌شود. سبک غالب Duck Typing است: «اگر مثل اردک راه برود...».

زیر PyTorch، محاسبات واقعی تانسور در کتابخانه‌های C++ و CUDA انجام می‌شود که از قبل کامپایل شده‌اند. Python فقط از طریق یک لایه‌ی اتصال (binding) آن‌ها را صدا می‌زند. این همان الگوی «زبان چسب» (glue language) است: سهولت زبان سطح بالا، سرعت زبان سطح پایین.

JSON قرارداد بین دو دنیای نوع است و JSX و CSS زبان‌های اعلانی‌اند: می‌گوییم «چه» می‌خواهیم، نه «چطور».

پرسش‌های احتمالی

س: Strong و Dynamic با هم تناقض ندارند؟

ج: نه؛ دو محور مستقل‌اند. Dynamic یعنی نوع در زمان اجرا بررسی می‌شود؛ Strong یعنی تبدیل ضمنی نـاامن انجـام نمی‌شـود. Python پویا و قوی است، JavaScript پویا و ضعیف‌تر (مثلاً "1" + 1 = "11").

س: Structural typing یعنی چه؟

ج: سازگاری بر اساس شکل نوع است نه نام آن؛ هر شیئی که فیلدهای Prediction را داشته باشد، Prediction حساب می‌شود. در مقابل Nominal typing مثل Java.

س: Python مفسری است یا کامپایلری؟

ج: هر دو: کد به bytecode کامپایل و روی ماشین مجازی CPython تفسیر می‌شود. TypeScript به JavaScript ترجمه می‌شود و موتور JS آن را با JIT اجرا می‌کند.

اسلاید ۱۹: PyTorch: یک زبان تخصصی درون Python

زمان پیشنهادی: ۱:۱۵ دقیقه

> **پیام اصلی:** PyTorch زبان جدیدی نیست؛ یک DSL درون‌زبانی است که از نحو خود Python استفاده می‌کند.

روی اسلاید

- Embedded DSL: نحو Python + Operator Overloading روی تانسورها.
- Define-by-run: گراف محاسبه هم‌زمان با اجرای کد ساخته می‌شود.
- Autograd: مشتق‌گیری خودکار معکوس روی همین گراف.
- Eager مثل مفسر؛ torch.compile مثل کامپایلر JIT.

متن گوینده

یک زبان تخصصی دامنه یا DSL، زبانی است برای یک دامنه‌ی خاص؛ مثل SQL برای داده یا Regex برای الگوی متن. دو نوع دارد: مستقل (external) با نحو و پارسر خودش، و درون‌زبانی (embedded) که داخل یک زبان میزبان ساخته می‌شود. PyTorch یک DSL درون‌زبانی در Python است.

ببینید تابع forward مدل من چقدر شبیه Python عادی اسـت: self.pool1(torch.relu(self.conv1(image))). هیچ نحـو جدیدی نیست. قدرت از Operator Overloading و فراخوانی اشیا می‌آید: وقتی می‌نویسیم model(image)، متـد ویـژه‌ی __call__ کلاس nn.Module اجرا می‌شود که hookها را مدیریت می‌کند و forward را صـدا می‌زنـد؛ یـک نمونـه از Dynamic Dispatch.

PyTorch سبک Define-by-run دارد: گراف محاسبه همان لحظه که کد اجرا می‌شود ساخته می‌شود. در مقابـل، TensorFlow 1.x سبک Define-and-run داشت: اول گراف را تعریف می‌کردی و بعد اجرا. مزیت define-by-run این اسـت کـه if و for معمولی Python کار می‌کنند و با print و دیباگر می‌شود خط به خط جلو رفت.

Autograd روی همین گراف ضبط‌شده مشتق‌گیری خودکار معکوس (reverse-mode) انجام می‌کند؛ وقتی loss.backward() را صدا می‌زنیم، گرادیان‌ها از انتها به ابتدا حساب می‌شوند. این همان پس‌انتشار خطاست که در اسلاید شـبکه‌ی عصـبی گفتم. در زمـان استنتاج، torch.inference_mode() ضبط گراف را خاموش می‌کند تا حافظه و زمان صرفه‌جویی شود.

اجرای eager شبیه مفسر است: هر عملگر جداگانه اجرا می‌شود. torch.compile در PyTorch 2 شبیه کامپایلر JIT است: TorchDynamo بایت‌کد Python را تحلیل و گراف را استخراج می‌کند و TorchInductor برای آن کد بهینه تولید می‌کند. همان مصالحه‌ی کلاسیک مفسر و کامپایلر: انعطاف و سادگی دیباگ در برابر سرعت.

پرسش‌های احتمالی

س: تفاوت DSL درون‌زبانی و مستقل چیست؟

ج: مستقل مثل SQL نحو و پارسر خودش را دارد؛ درون‌زبانی مثل PyTorch از نحو و ابزارهای زبان میزبان استفاده می‌کند و یاد گرفتنش ساده‌تر است، ولی به محدودیت‌های میزبان هم مقید است.

س: Operator Overloading در PyTorch کجاست؟

ج: عملگرهایی مثل + و * و @ روی Tensor با متدهای ویژه‌ای مثل __add__ و __matmul__ بازتعریف شده‌اند و هم محاسبه می‌کنند و هم در حالت گراد، گره‌ی گراف را ثبت می‌کنند.

س: Define-by-run چه عیبی دارد؟

ج: چون گراف هر بار در زمان اجرا ساخته می‌شود، بهینه‌سازی کلی سخت‌تر است؛ برای همین PyTorch 2 ابزار torch.compile را اضافه کرد تا گراف را بگیرد و کامپایل کند.

اسلاید ۲۰: سیستم نوع در مرز دو زبان

زمان پیشنهادی: ۱:۱۵ دقیقه

> **پیام اصلی:** نوع‌های TypeScript در زمان اجرا وجود ندارند؛ پس مرز شبکه را باید در زمان اجرا اعتبارسنجی کرد.

روی اسلاید

- "Literal Union: "Normal" | "Pneumonia"؛ فقط دو مقدار مجاز.
- as Prediction یک ادعا (assertion) است، نه بررسی؛ ناسالم (unsound).
- Type hint در Python اجرا نمی‌شود، اما FastAPI آن را با Reflection می‌خواند.
- شکل تانسور در سیستم نوع نیست: خطای ۱۵۶۸ فقط در زمان اجرا دیده می‌شود.

متن گوینده

در فرانت‌اند، نوع Prediction را تعریف کرده‌ام و فیلد prediction از نوع "Normal" | "Pneumonia" است؛ یک Literal Union Type. کامپایلر اجازه نمی‌دهد مقدار دیگری مثل "normal" با حرف کوچک بنویسم. این یعنی بخشی از دانش دامنه را در سیستم نوع رمز کرده‌ام.

اما جواب سرور را این‌طور خوانده‌ام: (await response.json() as Prediction. کلمه‌ی as یک Type Assertion است: به کامپایلر می‌گویم «به من اعتماد کن». هیچ بررسی‌ای در زمان اجرا انجام نمی‌شود، چون نوع‌ها پاک شده‌اند. اگر سرور شکل دیگری بفرستد، خطا جای دیگری و دیرتر ظاهر می‌شود. به همین دلیل سیستم نوع TypeScript عمداً ناسالم (unsound) طراحی شده: مصالحه بین ایمنی و عملی بودن.

در همان کد، شرط "detail" in result یک Type Narrowing است: کامپایلر بعد از این شرط می‌فهمد result از کدام شاخه‌ی Union است. این تحلیل جریان کنترل (control-flow analysis) در سیستم نوع است.

در Python، امضای predict(file: UploadFile = File(...)) -> dict[str, object] یک type hint است که مفسر اجرایش نمی‌کند. ولی FastAPI در زمان راه‌اندازی با Reflection این hintها را می‌خواند و از روی آن‌ها اعتبارسنجی ورودی و مستندات

OpenAPI می‌سازد. یعنی Annotation به داده‌ای تبدیل شده که رفتار برنامه را تعیین می‌کند. با from __future__ import annotations هم ارزیابی annotationها به تعویق می‌افتد (PEP 563).

model: PneumoniaCNN | None معادل Optional است؛ همان مسئله‌ی null safety. هر جا model را استفاده می‌کنم، اول None بودن را چک کرده‌ام.

و یک محدودیت جالب: عدد 32 * 7 * 7 = 1568 در لایه‌ی fc1 به اندازه‌ی ورودی ۲۸×۲۸ وابسته است. اگر ورودی را ۶۴×۶۴ کنم، کد بدون خطا اجرا می‌شود تا وقتی به ضرب ماتریسی برسد و خطای shape بدهد. شکل تانسور جزو سیستم نوع Python نیست. پژوهش‌هایی مثل Variadic Generics در PEP 646 و کتابخانه‌هایی مثل jaxtyping تلاش می‌کنند شکل تانسور را وارد نوع کنند؛ ایده‌ای نزدیک به نوع‌های وابسته (Dependent Types).

پرسش‌های احتمالی

س: Type erasure چه ضرری دارد؟

ج: نوع‌ها در زمان اجرا نیستند، پس داده‌ی بیرونی (مثل JSON سرور) بررسی نمی‌شود. as فقط کامپایلر را ساکت می‌کند. راه‌حل: اعتبارسنجی زمان اجرا مثل Zod یا تولید نوع از OpenAPI.

س: Sound بودن سیستم نوع یعنی چه؟

ج: یعنی اگر برنامه type-check شد، در زمان اجرا خطای نوع رخ نمی‌دهد. TypeScript عمداً unsound است (مثلاً به‌خاطر as و any) تا با کد JavaScript موجود سازگار بماند.

س: FastAPI چطور از type hint استفاده می‌کند وقتی Python آن را اجرا نمی‌کند؟

ج: با Reflection: هنگام ثبت مسیر، امضای تابع و annotationها را می‌خواند (مثل typing.get_type_hints) و بر اساس آن ورودی را اعتبارسنجی و OpenAPI تولید می‌کند.

س: می‌شود شکل تانسور را در نوع آورد؟

ج: تا حدی: (Variadic Generics) PEP 646 و کتابخانه‌هایی مثل jaxtyping اجازه می‌دهند ابعاد را در annotation بنویسیم و با ابزار بررسی کنیم؛ نسخه‌ی کامل آن به نوع‌های وابسته نیاز دارد.

اسلاید ۲۱: مفاهیم زبان در کد همین پروژه

زمان پیشنهادی: ۱:۱۵ دقیقه

> **پیام اصلی:** هر خط مهم این کد، یک مفهوم از درس طراحی زبان است.

متن گوینده

Decorator: @app.post("/predict") یعنی predict = app.post("/predict")(predict). دکوراتور یک تابع مرتبه‌بالاست که تابع دیگری را می‌گیرد و در جدول مسیرهای FastAPI ثبت می‌کند؛ نوعی فرابرنامه‌نویسی (metaprogramming).

Context Manager: with model_lock, torch.inference_mode(): دو منبع را با هم مدیریت می‌کند: قفل گرفته و حتماً آزاد می‌شود، و حالت بدون-گراد فعال و حتماً خاموش می‌شود، حتی اگر استثنا رخ دهد. این همان الگوی RAII در C++ است و پروتکل __enter__ و __exit__ آن را ممکن می‌کند. inference_mode در واقع یک حوزه‌ی پویا (dynamic scope) می‌سازد: رفتار هر عملگری که داخل این بلوک صدا زده شود عوض می‌شود.

async/await: await file.read(...) یک هم‌روال (coroutine) است که در حلقه‌ی رویداد منتظر می‌ماند و نخ را آزاد می‌کند. در جاوااسکریپت هم fetch یک Promise برمی‌گرداند و await قند نحوی (syntactic sugar) روی Promise است. هر دو هم‌روندی همکارانه (cooperative) هستند، نه موازی.

یک نقد صادقانه به کد خودم: استنتاج مدل کاری CPUبر است و داخل async def انجام می‌شود؛ یعنی حلقه‌ی رویداد در این مدت مسدود می‌شود. راه بهتر این است که predict را def معمولی کنم تا FastAPI آن را در threadpool اجرا کند، یا از run_in_threadpool استفاده کنم. آن‌وقت قفل model_lock واقعاً لازم می‌شود.

Closure: در useEffect فرانت‌اند، متغیر active توسط تابع پاک‌سازی بسته (capture) می‌شود. وقتی کامپوننت از صفحه می‌رود، active=false می‌شود و پاسخ‌های دیررسیده state را تغییر نمی‌دهند. این مثال کلاسیک بستار و چرخه‌ی عمر متغیر است.

ترکیب توابع: transforms.Compose([Resize, ToTensor]) دو تابع را به یک خط لوله تبدیل می‌کند: سبک تابعی. و زنجیره‌ی استثنا: raise HTTPException(...) from error علت اصلی خطا را حفظ می‌کند تا در لاگ دیده شود.

پرسش‌های احتمالی

س: Decorator دقیقاً معادل چیست؟

ج: @app.post("/predict") روی def predict معادل predict = app.post("/predict")(predict)است؛ app.post یک تابع برمی‌گرداند که تابع ما را ثبت و برمی‌گرداند.

س: Context manager چه فرقی با try/finally دارد؟

ج: از نظر معنایی همان است، اما الگو را در شیء کپسوله می‌کند (__enter__ و __exit__) تا تکرار نشود و فراموش نشود؛ شبیه RAII در C++.

س: async در Python موازی است؟

ج: نه؛ هم‌روند و همکارانه است. هر coroutine فقط در await نخ را رها می‌کند. کار CPUبر داخل async def حلقه‌ی رویداد را مسدود می‌کند؛ این ایراد کد فعلی است و باید به threadpool منتقل شود.

س: چرا Lock لازم است؟

ج: اگر درخواست‌ها در چند نخ اجرا شوند (مثلاً وقتی predict به def تبدیل شود و در threadpool برود)، قفل از دسترسی هم‌زمان به مدل جلوگیری می‌کند؛ در حالت فعلی async بیشتر جنبه‌ی احتیاط دارد.

اسلاید ۲۲: درس‌های طراحی زبان از این پروژه

زمان پیشنهادی: ۰:۴۵ دقیقه

> **پیام اصلی:** انتخاب زبان و سیستم نوع یک تصمیم طراحی است، نه سلیقه.

متن گوینده

چهار درس را جمع‌بندی می‌کنم. یک: هر لایه زبان خودش را می‌خواهد؛ پویایی Python برای آزمایش سریع مدل، ایستایی TypeScript برای رابطی که باید قابل اعتماد باشد، و C++ برای سرعت.

دو: نوع‌ها در مرز شبکه پاک می‌شوند، پس مرز را باید در زمان اجرا چک کرد؛ مثلاً با Pydantic در سرور و Zod در کلاینت، یا تولید خودکار نوع TypeScript از قرارداد OpenAPI تا یک منبع حقیقت داشته باشیم.

سه: یک DSL خوب نحو آشنای زبان میزبان را با معنای تخصصی ترکیب می‌کند؛ موفقیت PyTorch تا حد زیادی از همین است.

چهار: نوع‌ها می‌توانند ایمنی دامنه را رمز کنند. برچسب‌های تشخیص به‌صورت Literal Type، و در آینده شکل تانسور به‌صورت نوع. در نرم‌افزار پزشکی، هر خطایی که کامپایلر بگیرد، خطایی است که به بیمار نمی‌رسد.

پرسش‌های احتمالی

س: اگر دوباره می‌ساختی، زبان را عوض می‌کردی؟

ج: برای مدل نه؛ اکوسیستم Python بی‌رقیب است. ولی قرارداد API را با Pydantic تعریف و نوع TypeScript را خودکار تولید می‌کردم تا مرز دو زبان امن شود.

بخش: جمع‌بندی

اسلاید ۲۳: چالش‌ها، اخلاق و مقررات

زمان پیشنهادی: ۱:۰۰ دقیقه

> **پیام اصلی:** اعتبارسنجی مستقل و نظارت انسانی؛ اگر هوش مصنوعی اشتباه کند، چه کسی پاسخگوست؟

روی اسلاید

- رویکرد رایج: پزشک تصمیم‌گیرنده‌ی نهایی بماند.
- رضایت آگاهانه: آیا بیمار باید بداند AI در تشخیص او نقش داشته؟
- GDPR، HIPAA و EU AI Act یک قانون واحد نیستند.

متن گوینده

سوگیری الگوریتمی: الگوریتم‌ها از داده گذشته یاد می‌گیرند و نابرابری‌ها را تکرار می‌کنند. در پژوهش Obermeyer در Science (۲۰۱۹)، الگوریتمی در آمریکا هزینه درمان را معیار «نیاز» گرفته بود و چون بیماران سیاه‌پوست هزینه کمتری می‌کردند، نیازشان کمتر برآورد می‌شد. مدل‌های تشخیص ضایعه پوستی هم عمدتاً روی پوست روشن آموزش دیده‌اند و روی پوست تیره ضعیف‌ترند.

جعبه سیاه: شبکه‌های عمیق می‌گویند «چه»، ولی نمی‌گویند «چرا». پزشک نمی‌تواند چیزی را که درک نمی‌کند به بیمار توضیح دهد یا اشتباهش را کشف کند. هوش مصنوعی توضیح‌پذیر تلاشی برای حل این مسئله است؛ اما توضیح‌پذیری یک مفهوم واحد نیست و هیچ روشی به‌تنهایی صحت مدل را تضمین نمی‌کند.

فاصله آزمایشگاه تا بالین: مدل پیش‌بینی سپسیس شرکت Epic در ارزیابی مستقل منتشرشده در JAMA Internal Medicine (۲۰۲۱) بسیار ضعیف‌تر از ادعای سازنده عمل کرد؛ یعنی همان پایش و سپسیس که از آن گفتم نیازمند اعتبارسنجی مستقل است.

حریم خصوصی: داده سلامت حساس‌ترین نوع داده است و نشت آن قابل جبران نیست. یادگیری فدرال، که مدل بدون انتقال داده بیمار آموزش می‌بیند، در حال توسعه است؛ اما به‌تنهایی تضمین حریم خصوصی کامل نیست. و نابرابری دسترسی: اگر این فناوری فقط در کشورها و بیمارستان‌های ثروتمند باشد، شکاف سلامت بیشتر می‌شود.

مهم‌ترین پرسش اخلاقی: اگر هوش مصنوعی اشتباه کند، چه کسی پاسخگوست؟ پزشک، بیمارستان، شرکت سازنده یا برنامه‌نویس؟ در بیشتر کشورها هنوز پاسخ روشنی نیست و رویکرد رایج این است که پزشک تصمیم‌گیرنده نهایی بماند. پرسش دیگر رضایت آگاهانه است.

اتحادیه اروپا در «قانون هوش مصنوعی» (AI Act) هوش مصنوعی پزشکی را پرخطر دسته‌بندی کرده و شفافیت و نظارت انسانی را الزامی کرده است. قوانین حفاظت از داده مانند HIPAA در آمریکا و GDPR در اروپا هم حاکم‌اند.

نیازمند راستی‌آزمایی

> ⚠ Epic Sepsis (JAMA Internal Medicine ۲۰۲۱): جمعیت و نتایج دقیق از مقاله‌ی اصلی بررسی شود.
>
> ⚠ EU AI Act: از عبارت مطلق «همه‌ی AI پزشکی پرخطر است» پرهیز شود.

پرسش‌های احتمالی

س: اگر مدل اشتباه کند مسئولیت با کیست؟

ج: پاسخ حقوقی در بیشتر کشورها روشن نیست؛ رویکرد رایج این است که پزشک تصمیم‌گیرنده‌ی نهایی بماند. ابزار من صـریحاً بـرای تصمیم بالینی نیست.

اسلاید ۲۴: آیا هوش مصنوعی جای پزشک را می‌گیرد؟

زمان پیشنهادی: ۰:۳۰ دقیقه

> **پیام اصلی:** کمک به پزشک، نه ادعای جایگزینی مطلق.

روی اسلاید

- Eric Topol، Deep Medicine: پزشکی احتمالاً از سطح ۳ خودمختاری فراتر نمی‌رود.
- دمیس هسابیس: دقیق‌تر و سریع‌تر در تحلیل، اما نه جایگزین کامل.
- مهارت‌های ماشین‌ناپذیر: ارتباط، همدلی، رهبری و هوش هیجانی.

متن گوینده

اریک توپول در کتاب Deep Medicine از تمثیل خودروهای خودران استفاده می‌کند. خودرو پنج سطح خودمختاری دارد، از دستیار پارک در سطح ۱ تا سطح ۵ که انسان کاملاً حذف می‌شود. توپول معتقد است پزشکی احتمالاً از سطح ۳ فراتر نمی‌رود، چون نظارت انسانی حذف‌شدنی نیست. دمیس هسابیس هم گفته هوش مصنوعی ممکن است در تحلیل آزمایش و تصویر دقیق‌تر و سریع‌تر باشد، اما نمی‌تواند پزشک را کامل جایگزین کند. دو دیدگاه مستقل، یک نتیجه.

برای آینده پزشک، علاوه بر دانش پزشکی، آشنایی با علم داده، هوش مصنوعی و مسائل اخلاقی و حقوقی اهمیت می‌یابد، و مهارت‌هایی که ماشین ندارد، یعنی ارتباط، همدلی، رهبری و هوش هیجانی، ارزشمندتر می‌شوند.

تکمیلی: در ارائه از عبارت‌های مطلق مثل «هوش مصنوعی از پزشک دقیق‌تر است» پرهیز کنید، مگر اینکه ادعا دقیقاً بـه مطالعـه، وظیفه، جمعیت و معیار مشخص محدود شده باشد.

پرسش‌های احتمالی

س: بالاخره AI جای پزشک را می‌گیرد؟

ج: شواهد فعلی نقش کمکی را نشان می‌دهد؛ حتی در پروژه‌ی کوچک من، مدل بدون نظارت انسانی یک‌سوم افراد سالم را اشتباه برچسب می‌زد.

اسلاید ۲۵: جمع‌بندی

زمان پیشنهادی: ۰:۴۵ دقیقه

> **پیام اصلی:** داده‌ی مناسب، شواهد مستقل، پایش مداوم.

روی اسلاید

- هوش مصنوعی در تشخیص تصویری، پایش، کشف دارو و جراحی شواهد جدی دارد.
- هر ادعا باید روی داده‌ی مستقل بازآزمایی شود؛ پروژه‌ی خودم: ۹۶٪ روی Validation، ۸۵٪ روی Test.
- انتخاب زبان و سیستم نوع بخشی از ایمنی سیستم است؛ و هوش مصنوعی مکمل پزشک است، نه جایگزین.

متن گوینده

به آغاز برمی‌گردم. بیمار نیمه‌شب حالا شاید الگوریتمی کنارش بیدار است، اما تصمیم نهایی را باید پزشک بگیرد. سه نکته را جمع‌بندی می‌کنم. اول، هوش مصنوعی در تشخیص تصویری، پایش، کشف دارو و جراحی از مرحله وعده گذشته و شواهد جدی دارد، از MASAI تا halicin و AlphaFold. دوم، چالش‌هایی مثل سوگیری، جعبه سیاه، حریم خصوصی، مسئولیت قانونی و نابرابری دسترسی وجود دارد و هر ادعا باید در دنیای واقعی بازآزمایی شود. سوم، نقش هوش مصنوعی مکمل است، نه جایگزین.

در پروژه‌ی خودم همین را از نزدیک دیدم: مدلی که روی Validation ۹۶٪ دقت داشت، روی داده‌ی مستقل به ۸۵٪ رسید و یک‌سوم افراد سالم را اشتباه برچسب زد. مهندسی هوش مصنوعی پزشکی یعنی همین شکاف را جدی بگیریم.

و از دید این درس: سیستمی که ساختم نشان داد انتخاب زبان، DSL و سیستم نوع فقط موضوع سلیقه نیست؛ مرز بین Python و TypeScript جایی است که نوع‌ها پاک می‌شوند و باید آگاهانه اعتبارسنجی کنیم.

هوش مصنوعی جایگزین پزشک نمی‌شود، اما پزشکی که از آن استفاده کند، جایگزین پزشکی خواهد شد که استفاده نمی‌کند.

از توجه شما سپاسگزارم.

پرسش‌های احتمالی

س: مهم‌ترین چیزی که از این پروژه یاد گرفتی؟

ج: عدد Validation وعده‌ی دنیای واقعی نیست، و مرز بین زبان‌ها و نوع‌ها همان جایی است که باید بیشترین بررسی را داشته باشد.

بخش: پیوست‌ها

پیوست: پیوست الف: ادعاهایی که پیش از ارائه باید راستی‌آزمایی شوند

پیام اصلی: هیچ ادعای حساسی صرفاً به دلیل حضور در این ارائه، تأییدشده تلقی نشود.

متن گوینده

این موارد از متن اصلی حذف نشده‌اند؛ هدف این فهرست این است که قبل از ارائه بتوانی ادعاهای پرریسک را بررسی کنی و از ارائه‌ی عدد نادرست یا نتیجه‌گیری بیش از شواهد جلوگیری شود.

پیوست: پیوست ب: منابع و پیوندها

پیام اصلی: برای اسلایدهای نهایی، مقاله‌ی اصلی در اولویت است؛ تاریخ دسترسی و نسخه را ثبت کنید.

متن گوینده

منابع زیر نقطه‌ی شروع‌اند. برای هر عدد یا ادعای مربوط به محصول، تاریخ دسترسی و نسخه‌ی دقیق را ثبت کن.

پیوست: پیوست ج: مینی‌پروژه‌ی تمرینی PyTorch (Wisconsin)

پیام اصلی: نتیجه‌ی آموزشی است و محدودیت دارد؛ ابزار تشخیص بیمار نیست.

روی اسلاید

- ۵۶۹ نمونه، ۳۰ ویژگی عددی از هسته‌ی سلولی؛ کلاس مثبت: Malignant.
- Scaler فقط روی Train فیت شد تا اطلاعات Test نشت نکند.
- دو نمونه‌ی بدخیم، خوش‌خیم پیش‌بینی شدند (FN=2).

- Validation Loss ثبت نشده؛ درباره‌ی نبود overfitting نمی‌توان نتیجه گرفت.

متن گوینده

داده: Breast Cancer Wisconsin Diagnostic از scikit-learn؛ ۵۶۹ نمونه و ۳۰ ویژگی عددی که از اندازه‌گیری‌های هسته‌ی سلولی استخراج شده‌اند، نه پیکسل‌های تصویر خام. تقسیم داده: ۸۰٪ آموزش و ۲۰٪ آزمون، با stratify و random_state=42؛ مجموعه‌ی آزمون شامل ۱۱۴ نمونه بود.

شبکه: Linear(30,16) → ReLU → Linear(16,8) → ReLU → Linear(8,1)؛ خروجی یک logit است و با sigmoid به احتمال تبدیل می‌شود. Loss: BCEWithLogitsLoss که ترکیب عددی پایدارتری از sigmoid و binary cross-entropy است. بهینه‌ساز Adam با ۱۰۰ epoch، batch size ۳۲، lr=0.001.

تفسیر ماتریس خطا: TP=40، FN=2، FP=0، TN=72. دو نمونه‌ی بدخیم در این مجموعه‌ی آزمون به‌عنوان خوش‌خیم پیش‌بینی شده‌اند. به همین دلیل، با وجود Accuracy بالا، بررسی حساسیت و تعداد منفی کاذب مهم است.

ROC-AUC حدود 0.995 به معنی رتبه‌بندی بسیار خوب روی همین مجموعه‌ی آزمون است؛ این عدد نه کالیبراسیون احتمال‌ها را اثبات می‌کند، نه عملکرد در بیمارستان دیگر را، و نه مجوز استفاده‌ی بالینی است. تعداد آزمون کم است و نتیجه ممکن است با تقسیم‌بندی دیگری تغییر کند.

آزمایش بعدی: احتمال‌ها را با thresholdهای 0.3، 0.5 و 0.7 به برچسب تبدیل و Sensitivity/Specificity را مقایسه کنید؛ آستانه باید با داده‌ی validation انتخاب شود، نه با تنظیم مکرر روی test.

پیوست: پیوست د: پایش و پیش‌بینی خطر

> **پیام اصلی:** زمان پیش‌بینی، نشت داده و هشدار کاذب را جدی بگیریم.

روی اسلاید

- ترکیب داده‌ی چند دستگاه برای یافتن وضعیت‌هایی مثل سپسیس.
- تخمین خطر بیماری قلبی یا دیابت از سوابق و آزمایش‌ها.
- ساعت هوشمند: نامنظمی ضربان و فیبریلاسیون دهلیزی.
- هشدار بیش از حد ← Alert Fatigue.

متن گوینده

همان بیمار نیمه‌شب را به یاد بیاورید. مانیتورها علائم حیاتی را ثبت می‌کنند، اما هوش مصنوعی می‌تواند داده‌های چند دستگاه را کنار هم بگذارد و دنبال وضعیت‌های پیچیده‌ای مثل سپسیس بگردد، که تشخیص زودهنگامش جان بیمار را نجات می‌دهد. با بررسی سوابق، آزمایش‌ها و عوامل خطر، این سامانه‌ها احتمال بیماری‌های قلبی یا دیابت را هم تخمین می‌زنند. در سطح جامعه، ساعت‌های هوشمند می‌توانند نامنظمی ضربان و فیبریلاسیون دهلیزی را تشخیص دهند.

باید مشخص شود مدل در چه زمانی پیش‌بینی را تولید می‌کند، پیامد در چه بازه‌ای تعریف شده و کدام داده‌ها واقعاً تا لحظه‌ی پیش‌بینی در دسترس بوده‌اند. اگر ویژگی‌هایی پس از شروع بیماری یا پس از تصمیم درمانی وارد مدل شوند، خطر data leakage وجود دارد.

ارزیابی فقط با AUC کافی نیست؛ زمان هشدار، نرخ هشدار کاذب، حساسیت در آستانه‌ی عملیاتی و اثر بر تصمیم بالینی مهم‌اند. اعتبارسنجی بیرونی و پایش Distribution Shift ضروری‌اند.

پیوست: پیوست ه: درمان، جراحی و مدیریت اطلاعات

> **پیام اصلی:** پزشکی تقویت‌شده: ربات دقت را بالا می‌برد، جراح کنترل را در دست دارد.

روی اسلاید

- انتخاب درمان: مقایسه‌ی روش‌ها بر اساس اطلاعات بیمار و پژوهش‌ها.
- Flok Health (بریتانیا): فیزیوتراپی مبتنی بر AI با ویزیت ویدیویی فوری.
- da Vinci: عمدتاً خودمختار نیست؛ جراح کنسول را کنترل می‌کند.
- بخش اداری: خلاصه‌ی پرونده و پیش‌نویس گزارش ویزیت.

متن گوینده

در انتخاب درمان، هوش مصنوعی اطلاعات بیمار و نتایج پژوهش‌ها را بررسی می‌کند و به پزشک در مقایسه روش‌ها کمک می‌کند. مثال ملموس Flok Health در بریتانیاست؛ کلینیک فیزیوتراپی مبتنی بر هوش مصنوعی که با ویزیت ویدیویی فوری، زمان انتظار درمان کمردرد و مشکلات اسکلتی-عضلانی را تا نصف کاهش داده است.

در جراحی، دا وینچی رایج‌ترین ربات است و نکته جالب اینکه عمدتاً خودمختار نیست: جراح کنسول را کنترل می‌کند و ربات دقت و ثبات حرکت را بالا می‌برد. نتیجه، برش‌های کوچک‌تر، خونریزی کمتر و بهبودی سریع‌تر است. این همان «پزشکی تقویت‌شده» است. در بخش اداری هم هوش مصنوعی می‌تواند پرونده‌ها را خلاصه کند و پیش‌نویس گزارش ویزیت بنویسد.

نیازمند راستی‌آزمایی

> ⚠️ Flok Health و «کاهش زمان انتظار تا نصف»: منبع رسمی و شاخص دقیق بررسی شود.
>
> ⚠️ da Vinci: از تعمیم نتایج به همه‌ی جراحی‌ها پرهیز شود.

۴. پروژه‌ی PneumoniaCNN با جزئیات کامل

۴.۱ داده

داده‌ی پروژه PneumoniaMNIST از مجموعه‌ی MedMNIST v2 است. این داده از دیتاست رادیوگرافی قفسه سینه‌ی کودکان (Kermany و همکاران) ساخته شده: تصاویر خاکستری از وسط به‌صورت مربعی برش خورده و به ۱×۲۸×۲۸ کوچک شده‌اند. مجموع ۵۸۵۶ تصویر است. Train اصلی به نسبت ۹ به ۱ به Train و Validation تقسیم شده و مجموعه‌ی آزمون اصلی به‌عنوان Test استفاده می‌شود.

بخش	کل	نرمال	پنومونی	توضیح
Train	4708	—	—	۹۰٪ از Train اصلی
Validation	524	135	389	۱۰٪ از Train اصلی؛ برای انتخاب آستانه
Test	624	234	390	مجموعه‌ی جداگانه؛ فقط یک بار برای گزارش

نسبت پنومونی به نرمال در Validation حدود ۳۸۹/۱۳۵ ≈ ۲٫۹ است. به همین دلیل Accuracy به‌تنهایی گمراه‌کننده است: مدلی که همیشه بگوید «پنومونی» روی Validation به ۷۴٪ دقت می‌رسد (۳۸۹/۵۲۴) و روی Test به ۶۲٫۵٪ (۳۹۰/۶۲۴).

> **نکته‌ی امتحانی:** این «خط پایه‌ی کلاس غالب» را حفظ کن. اگر کسی گفت ۸۵٪ دقت زیاد است، بگو خط پایه‌ی بی‌هوش روی Test ۶۲٫۵٪ است و مدل حدود ۲۲ واحد درصد بالاتر است.

۴.۲ مدل

```
class PneumoniaCNN(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(32 * 7 * 7, 64)
        self.fc2 = nn.Linear(64, 2)

    def forward(self, image: torch.Tensor) -> torch.Tensor:
        image = self.pool1(torch.relu(self.conv1(image)))
        image = self.pool2(torch.relu(self.conv2(image)))
        image = torch.flatten(image, start_dim=1)
        image = torch.relu(self.fc1(image))
        return self.fc2(image)
```

ابعاد در هر لایه

لایه	خروجی	توضیح
ورودی	1×28×28	یک کانال خاکستری
Conv1 + ReLU	16×28×28	۱۶ فیلتر ۳×۳؛ padding=1 اندازه را ثابت نگه می‌دارد
MaxPool	16×14×14	پنجره‌ی ۲×۲، گام ۲؛ نصف شدن ابعاد
Conv2 + ReLU	32×14×14	۳۲ فیلتر ۳×۳ روی ۱۶ کانال

لایه	خروجی	توضیح
MaxPool	32×7×7	نصف شدن دوباره
Flatten	1568	۳۲ × ۷ × ۷
fc1 + ReLU	64	لایه‌ی کاملاً متصل
fc2	2	logit نرمال و پنومونی؛ Softmax ← احتمال

شمارش پارامترها

لایه	محاسبه	تعداد
Conv1	16×(3×3×1) + 16	160
Conv2	32×(3×3×16) + 32	4,640
fc1	64×1568 + 64	100,416
fc2	2×64 + 2	130
جمع		105,346

حدود ۹۵٪ پارامترها در fc1 است. اندازه‌ی فایل مدل (حدود ۴۲۵ کیلوبایت) با ۱۰۵٬۳۴۶ عدد ۴ بایتی (float32) جور درمی‌آید. ResNet-18 حدود ۱۱ میلیون پارامتر دارد؛ یعنی مدل تو حدود یک درصد آن است.

تصمیم‌های آموزش

- **Loss وزن‌دار** (در checkpoint ذخیره شده: class_weighted_loss = True): خطای کلاس کم‌تعداد (نرمال) وزن بیشتری می‌گیرد تا مدل با گفتن همیشگی «پنومونی» به Loss کم نرسد.
- **آستانه‌ی ۰٫۳** (در checkpoint: threshold = 0.3): اگر احتمال پنومونی ≥ ۰٫۳ باشد برچسب پنومونی داده می‌شود. این انتخاب روی Validation انجام شده تا Sensitivity بالا بماند.
- **خروجی دوتایی با Softmax** به‌جای یک خروجی با Sigmoid: از نظر ریاضی معادل‌اند؛ دو خروجی با CrossEntropy کار با وزن کلاس را ساده می‌کند و به چندکلاسه تعمیم می‌یابد.

۴.۳ انتخاب آستانه روی Validation

آستانه	Accuracy	Sensitivity	Specificity	FP	FN
0.3	96.4%	97.4%	93.3%	9	10
0.4	95.6%	95.4%	96.3%	5	18
0.5	95.0%	94.1%	97.8%	3	23
0.6	93.3%	91.5%	98.5%	2	33
0.7	91.4%	88.7%	99.3%	1	44

این جدول روی ۵۲۴ تصویر Validation است (۱۳۵ نرمال، ۳۸۹ پنومونی). می‌توانی هر ردیف را بازسازی کنی؛ مثلاً برای ۰٫۳: Sensitivity = (۳۸۹ − ۱۰)/۳۸۹ = ۰٫۹۷۴ و Specificity = (۱۳۵ − ۹)/۱۳۵ = ۰٫۹۳۳ و Accuracy = (۳۷۹ + ۱۲۶)/۵۲۴ = ۰٫۹۶۴.

مراقب این اشتباه باش: ماتریس Validation که در خروجی خودت گزارش شده ([[132, 3], [23, 366]]) با آستانه‌ی ۰٫۵ است، نه ۰٫۳؛ چون FP=3 و FN=23 دقیقاً ردیف ۰٫۵ جدول است. در ارائه برای مقایسه‌ی منصفانه، ماتریس Validation با آستانه‌ی ۰٫۳ (TN=126، FP=9، FN=10، TP=379) را آورده‌ام.

۴.۴ نتایج نهایی روی Test (آستانه ۰٫۳)

	پیش‌بینی نرمال	پیش‌بینی پنومونی	جمع
واقعاً نرمال	TN = 148	FP = 86	234
واقعاً پنومونی	FN = 8	TP = 382	390

معیار	فرمول	محاسبه	مقدار
Accuracy	N/(TP+TN)	624/(382+148)	84.9%
Sensitivity (Recall پنومونی)	TP/(TP+FN)	382/390	97.9%
Specificity (Recall نرمال)	TN/(TN+FP)	148/234	63.2%
Precision پنومونی	TP/(TP+FP)	382/468	81.6%
Precision نرمال	TN/(TN+FN)	148/156	94.9%
F1 پنومونی	2PR/(P+R)		0.890
F1 نرمال			0.759
Macro F1	میانگین ساده		0.825
Weighted F1	میانگین وزن‌دار		0.841

مقایسه‌ی Validation و Test (هر دو با آستانه‌ی ۰٫۳)

معیار	Validation	Test	تغییر
Accuracy	96.4%	84.9%	−11.5
Sensitivity	97.4%	97.9%	≈ ثابت
Specificity	93.3%	63.2%	−30.1
Precision پنومونی	97.7%	81.6%	−16.1

چرا این شکاف؟ (این را عمیق بلد باش)

- **Distribution Shift:** Validation از دل همان Train اصلی آمده و شبیه آن است؛ Test مجموعه‌ای جداست که جداگانه گردآوری شده. مدل روی توزیع آشنا خوب است و روی توزیع کمی متفاوت ضعیف‌تر.
- **احتمال نشت در سطح بیمار:** از هر بیمار ممکن است چند تصویر باشد. اگر تقسیم Train/Validation بر اساس تصویر بوده و نه بیمار، Validation خوش‌بینانه می‌شود.
- **آستانه‌ی متمایل به پنومونی:** آستانه‌ی ۰٫۳ عمداً مدل را حساس کرده؛ روی توزیع جدید این تمایل به‌صورت مثبت کاذب زیاد ظاهر شده.

- **وضوح پایین:** در ۲۸×۲۸ تفاوت‌های ظریف ریه‌ی سالم و بیمار از بین می‌رود و مرز تصمیم شکننده‌تر است.

مقایسه با بنچمارک منتشرشده

روش (گزارش MedMNIST v2)	Test ACC	Test AUC
PneumoniaCNN (پروژه‌ی من)	0.849	—
ResNet-18 (28)	0.854	0.944
ResNet-18 (224)	0.864	0.956
ResNet-50 (224)	0.884	0.962
AutoKeras	0.878	0.947
Google AutoML Vision	0.946	0.991

نتیجه: مدل کوچک تو با حدود یک درصد پارامترهای ResNet-18 تقریباً به همان دقت روی همین Test رسیده. AUC مدل خودت را حساب نکرده‌ای؛ اگر کسی پرسید، صادقانه بگو در گزارش تو نیست و قدم بعدی است.

۴.۵ سیستم نرم‌افزاری

لایه	فناوری	مسئولیت
رابط	Next.js 16 · React 19 · TypeScript	بارگذاری تصویر (کشیدن، انتخاب، Ctrl+V، نمونه‌ها)، نمایش احتمال‌ها، آستانه و هشدار نزدیک به آستانه، صفحه‌ی ارائه
API	FastAPI · Uvicorn	health/ برای وضعیت، predict/ برای پیش‌بینی؛ کنترل نوع فایل (JPG/PNG) و حجم (۱۰ مگابایت)، CORS
مدل	PyTorch · torchvision · Pillow	تبدیل به خاکستری، تغییر اندازه به ۲۸×۲۸، ToTensor (بازه‌ی ۰ تا ۱)، Softmax و مقایسه با آستانه

ناهماهنگی آموزش و سرویس (training–serving skew): PneumoniaMNIST تصویر را اول از وسط مربعی برش داده و بعد کوچک کرده؛ سرور فقط تغییر اندازه می‌دهد. برای تصاویر غیرمربعی شکل ریه کمی کشیده می‌شود. اگر پرسیدند، این را به‌عنوان یک بهبود مشخص بگو: همان CenterCrop در سرور اعمال شود. همچنین اگر در آموزش Normalize داشته‌ای، باید در سرور هم باشد.

۴.۶ محدودیت‌ها و قدم بعدی

- داده‌ی کودکان است؛ تعمیم به بزرگسالان سنجیده نشده.
- وضوح ۲۸×۲۸ جزئیات ریز ریه را از بین می‌برد.
- Specificity روی Test فقط ۶۳٪ است؛ هشدار کاذب زیاد.
- تقسیم Train/Validation در سطح بیمار تضمین نشده است.
- احتمال‌ها کالیبره نشده‌اند؛ ۹۹٪ یعنی «مطمئن»، نه «درست».

قدم بعدی

- ورودی ۲۲۴×۲۲۴ و Transfer Learning (مثلاً ResNet).
- Data Augmentation برای تعمیم بهتر.

- تقسیم داده بر اساس بیمار و اعتبارسنجی بیرونی.
- Grad-CAM برای دیدن ناحیه‌ی مؤثر در تصمیم.
- کالیبراسیون (Temperature Scaling) و آستانه‌ی متعادل‌تر.

۵. مفاهیم طراحی زبان با جزئیات کامل

این بخش هسته‌ی ارتباط ارائه با درس است. هر مفهوم با مثالی از کد واقعی پروژه آمده تا اگر استاد پرسید «کجای کدت؟» بتوانی دقیق نشان بدهی.

۵.۱ نقشه‌ی زبان‌های پروژه

زبان	نقش	سیستم نوع	مدل اجرا
TypeScript / TSX	رابط کاربری	ایستا، ساختاری، تدریجی	ترجمه به JavaScript؛ نوع‌ها پاک می‌شوند؛ موتور JS با JIT اجرا می‌کند
Python	API و هماهنگی	پویا، قوی، Duck typing	کامپایل به bytecode، تفسیر روی ماشین مجازی CPython
PyTorch (DSL)	تعریف و اجرای مدل	Tensor؛ شکل فقط در زمان اجرا	eager (شبیه مفسر) یا torch.compile (شبیه JIT)
C++ / CUDA	محاسبه‌ی تانسور	ایستا، اسمی	از پیش کامپایل‌شده؛ از Python از طریق binding صدا زده می‌شود
JSON	قرارداد بین کلاینت و سرور	بدون طرح‌واره‌ی اجباری	داده
JSX / CSS	ساختار و ظاهر رابط	—	اعلانی

محورهای سیستم نوع (حتماً بلد باش)

محور	تعریف	در پروژه
ایستا / پویا	نوع در زمان کامپایل بررسی شود یا در زمان اجرا	TypeScript ایستا، Python پویا
قوی / ضعیف	آیا تبدیل ضمنی ناامن انجام می‌شود	Python قوی ('1' + 1 خطا)؛ JavaScript ضعیف‌تر ('1' + 1 = '11')
ساختاری / اسمی	سازگاری بر اساس شکل یا نام نوع	TypeScript ساختاری؛ C++ و Java اسمی
سالم / ناسالم	آیا type-check عدم خطای نوع در اجرا را تضمین می‌کند	TypeScript عمداً ناسالم (as و any)
تدریجی	ترکیب بخش‌های دارای نوع و بی‌نوع	TypeScript و type hint در Python

۵.۲ PyTorch به‌عنوان DSL درون‌زبانی

- **DSL** زبانی برای یک دامنه‌ی خاص است. **مستقل (external)** مثل SQL نحو و پارسر خودش را دارد؛ **درون‌زبانی (embedded)** مثل PyTorch داخل زبان میزبان ساخته می‌شود و از نحو، ابزارها و کتابخانه‌های آن استفاده می‌کند.
- **Operator Overloading:** روی Tensor عملگرهایی مثل + و * و @ با متدهای ویژه (__mul__، __add__، __matmul__) بازتعریف شده‌اند.
- **فراخوانی شیء:** `model(image)` متد `__call__` کلاس nn.Module را اجرا می‌کند که hookها را مدیریت و forward را صدا می‌زند؛ این Dynamic Dispatch است چون forward کلاس فرزند اجرا می‌شود.

- **Define-by-run:** گراف محاسبه هم‌زمان با اجرا ساخته می‌شود؛ پس if و for و print معمولی Python کار می‌کنند. در مقابل **Define-and-run** در TensorFlow 1.x که اول گراف تعریف و بعد در Session اجرا می‌شد.
- **Autograd:** مشتق‌گیری خودکار معکوس (reverse-mode). هر عملگر روی تانسور با requires_grad یک گره در گراف ثبت می‌کند؛ `loss.backward()` گرادیان‌ها را با قاعده‌ی زنجیره‌ای از انتها به ابتدا حساب می‌کند.
- **inference_mode:** با `with torch.inference_mode():` ضبط گراف خاموش می‌شود؛ صرفه‌جویی حافظه و زمان در استنتاج.
- **Eager در برابر torch.compile:** eager هر عملگر را جدا اجرا می‌کند (شبیه مفسر)؛ در PyTorch 2، torch.compile با TorchDynamo بایت‌کد Python را تحلیل و گراف را استخراج می‌کند و TorchInductor کد بهینه تولید می‌کند (شبیه کامپایلر JIT). مصالحه: سادگی دیباگ و انعطاف در برابر سرعت و هزینه‌ی کامپایل.

۵.۳ سیستم نوع در مرز دو زبان

سمت TypeScript

```
type Prediction = {
  prediction: "Normal" | "Pneumonia";      // literal union
  label: string;
  confidence: number;
  probabilities: { Normal: number; Pneumonia: number };
  threshold: number;
  model: string;
};

const result = (await response.json()) as Prediction | { detail?: string };
if (!response.ok) {
  throw new Error("detail" in result && result.detail ? result.detail : "...");
}
setPrediction(result as Prediction);
```

- **Literal Union Type:** فقط دو رشته‌ی مشخص مجازند؛ دانش دامنه در نوع رمز شده.
- **Type Assertion (as):** به کامپایلر می‌گوید «به من اعتماد کن». در زمان اجرا هیچ بررسی‌ای نیست چون TypeScript نوع‌ها را پاک می‌کند (Type Erasure).
- **Narrowing با in:** `"detail" in result` باعث می‌شود کامپایلر بفهمد result از کدام شاخه‌ی Union است؛ تحلیل جریان کنترل در سیستم نوع.
- **Generics:** `useState<File | null>(null)` و `Record<IconName, ReactNode>` نمونه‌های نوع پارامتری‌اند.

سمت Python

```
from __future__ import annotations      # PEP 563: postponed evaluation

model: PneumoniaCNN | None = None        # PEP 604 union = Optional

@app.post("/predict")
async def predict(file: UploadFile = File(...)) -> dict[str, object]:
    if model is None:
        raise HTTPException(status_code=503, detail="...")
```

- **Type hint اجرا نمی‌شود:** مفسر CPython annotationها را فقط ذخیره می‌کند.
- **Reflection در FastAPI: FastAPI** امضای تابع و annotationها را می‌خواند و از روی آن ورودی را اعتبارسنجی و مستند OpenAPI تولید می‌کند؛ annotation به داده‌ای تبدیل شده که رفتار را تعیین می‌کند.
- **Null safety:** `PneumoniaCNN | None` یعنی ممکن است مدل بارگذاری نشده باشد؛ هر جا استفاده شده، اول None بررسی شده.
- **شکل تانسور خارج از نوع:** `nn.Linear(32 * 7 * 7, 64)` به ورودی ۲۸×۲۸ وابسته است؛ با ورودی دیگر، خطای ضرب ماتریس فقط در زمان اجرا رخ می‌دهد. (Variadic Generics، Python 3.11) PEP 646 و کتابخانه‌هایی مثل jaxtyping تلاش می‌کنند ابعاد را در annotation بیاورند؛ حالت کامل آن به نوع‌های وابسته (Dependent Types) نزدیک است.

> **پیشنهاد بهبود:** قرارداد API را با یک مدل Pydantic تعریف کن (response_model)، از OpenAPI نوع‌های TypeScript را خودکار تولید کن و در کلاینت با Zod در زمان اجرا اعتبارسنجی کن. این یعنی یک منبع حقیقت برای دو سیستم نوع.

۵.۴ ویژگی‌های زبانی در کد پروژه

۱. Decorator — تابع مرتبه‌بالا، فرابرنامه‌نویسی

```
@app.post("/predict")
async def predict(file): ...
# == predict = app.post("/predict")(predict)
```

app.post یک تابع برمی‌گرداند که تابع ما را در جدول مسیرها ثبت می‌کند و همان را برمی‌گرداند.

۲. Context Manager — مدیریت منبع، شبیه RAII؛ حوزه‌ی پویا

```
with model_lock, torch.inference_mode():
    logits = model(image)
    probabilities = torch.softmax(logits, dim=1)[0]
```

پروتکل __enter__/__exit__ تضمین می‌کند قفل آزاد و حالت بدون‌گراد خاموش شود، حتی با استثنا. معادل try/finally ولی کپسوله و قابل استفاده‌ی مجدد.

۳. async/await — هم‌روال، حلقه‌ی رویداد

```
contents = await file.read(MAX_IMAGE_BYTES + 1)

const response = await fetch(`${API_URL}/predict`, {...});
```

در Python هم‌روال در await نخ را آزاد می‌کند؛ در JS، await قند نحوی روی Promise است. هر دو هم‌روندی همکارانه‌اند، نه موازی.

۴. Closure — بستار، چرخه‌ی عمر

```
useEffect(() => {
  let active = true;
  const run = () => { if (active) void checkHealth(); };
  const id = window.setInterval(run, 6000);
  return () => { active = false; window.clearInterval(id); };
```

```
}, [checkHealth]);
```

تابع پاک‌سازی متغیر active را capture می‌کند؛ بعد از unmount پاسخ‌های دیررس state را تغییر نمی‌دهند.

۵. Function Composition — سبک تابعی

```
preprocess = transforms.Compose([
    transforms.Resize((28, 28)),
    transforms.ToTensor(),
])
```

چند تابع کوچک به یک خط لوله تبدیل شده‌اند؛ ترتیب اجرا از چپ به راست.

۶. Exception Chaining — مدیریت خطا

```
except (UnidentifiedImageError, OSError) as error:
    raise HTTPException(status_code=400, detail="...") from error
```

با from، استثنای اصلی در __cause__ نگه داشته می‌شود و در لاگ دیده می‌شود.

۷. OOP و وراثت — شیءگرایی، Dynamic Dispatch

```
class PneumoniaCNN(nn.Module):
    def forward(self, image): ...
```

کلاس از nn.Module ارث می‌برد؛ __call__ پدر، forward فرزند را صدا می‌زند (الگوی Template Method).

۵.۵ نقد صادقانه‌ی کد: async و کار CPUبر

در کد فعلی، استنتاج مدل داخل `async def predict` انجام می‌شود. استنتاج کاری CPUبر است و await ندارد؛ پس در این مدت حلقه‌ی رویداد مسدود است و درخواست‌های دیگر (حتی /health) منتظر می‌مانند. دو راه اصلاح:

```
# 1) def معمولی: FastAPI آن را در threadpool اجرا می‌کند
@app.post("/predict")
def predict(file: UploadFile = File(...)) -> dict[str, object]:
    contents = file.file.read(MAX_IMAGE_BYTES + 1)
    ...

# 2) async نگه داشتن و انتقال بخش سنگین
from starlette.concurrency import run_in_threadpool
result = await run_in_threadpool(run_inference, image)
```

در هر دو حالت، چند نخ ممکن است هم‌زمان به مدل دسترسی پیدا کنند و آن‌وقت `model_lock` واقعاً لازم می‌شود. در CPython یک GIL وجود دارد، اما عملیات سنگین PyTorch در C++ اجرا می‌شوند و GIL را آزاد می‌کنند؛ برای همین استفاده از چند نخ برای استنتاج معنا دارد.

۵.۶ مدیریت حافظه (اگر پرسیدند)

- **CPython:** شمارش ارجاع (Reference Counting) به‌علاوه‌ی جمع‌آوری زباله‌ی چرخه‌ای (Cyclic GC). شیء با صفر شدن ارجاع‌ها فوراً آزاد می‌شود.
- **موتور V8 در JavaScript:** جمع‌آوری زباله‌ی ردیابی‌کننده و نسلی (Generational, Tracing GC).
- **تانسورهای PyTorch:** حافظه‌ی داده در C++ مدیریت می‌شود و با شمارش ارجاع شیء Python آزاد می‌شود. URL.createObjectURL در فرانت‌اند هم باید با revokeObjectURL آزاد شود؛ که در کد انجام شده (مدیریت دستی منبع در زبانی با GC).

۵.۷ چهار درس طراحی زبان

1. هر لایه، زبان خودش: پویایی Python برای آزمایش مدل، ایستایی TypeScript برای رابط قابل اعتماد، C++ برای سرعت.
2. مرز را در زمان اجرا چک کن: نوع‌ها در JSON پاک می‌شوند؛ Pydantic در سرور، Zod در کلاینت، یا تولید نوع از OpenAPI.
3. DSL خوب = نحو آشنا + معنای تخصصی: PyTorch بدون نحو جدید، با Operator Overloading و Autograd، زبان یادگیری عمیق شد.
4. نوع، ایمنی دامنه را رمز می‌کند: برچسب تشخیص به‌صورت Literal Type؛ در آینده شکل تانسور به‌صورت نوع.

۶. بانک پرسش و پاسخ

پرسش‌های اسلایدها به‌علاوه‌ی پرسش‌های عمومی که ممکن است استاد بپرسد. پاسخ‌ها را کوتاه و با عدد بگو.

بخش	پرسش	پاسخ کوتاه
عنوان و سؤال آغازین	این موضوع چه ربطی به درس طراحی زبان دارد؟	پروژه یک سیستم چندزبانه است: Python، TypeScript، و C++ زیر PyTorch. در بخش آخر نشان می‌دهم PyTorch یک DSL درون‌زبانی است و سیستم نوع در مرز زبان‌ها چه نقشی دارد.
چرا پزشکی از AI استفاده می‌کند؟	اگر داده زیاد است، چرا فقط آمار کلاسیک کافی نیست؟	برای داده‌ی جدولی گاهی کافی است؛ اما برای تصویر خام، ویژگی‌ها باید از پیکسل‌ها یاد گرفته شوند و این کار شبکه‌های کانولوشنی است.
AI، ML و DL	فرق یادگیری ماشین با برنامه‌نویسی معمولی چیست؟	در برنامه‌نویسی معمولی قاعده را ما می‌نویسیم؛ در یادگیری ماشین قاعده (وزن‌ها) از روی داده و برچسب استخراج می‌شود.
معماری سامانه	Data leakage در پروژه‌ی خودت ممکن است رخ داده باشد؟	ممکن است: Validation از دل Train اصلی جدا شده و تضمینی نیست که تصاویر یک بیمار در هر دو نباشند. Test اما مجموعه‌ی جداست؛ برای همین Test را معیار اصلی گزارش کردم.
شبکه‌ی عصبی به زبان ساده	چرا Adam و نه SGD؟	Adam نرخ یادگیری را برای هر پارامتر تطبیق می‌دهد و معمولاً بدون تنظیم زیاد سریع همگرا می‌شود؛ برای پروژه‌ی آموزشی انتخاب امن‌تری است.
شبکه‌ی عصبی به زبان ساده	Backpropagation چه ربطی به Autograd دارد؟	Backpropagation همان مشتق‌گیری خودکار معکوس روی گراف شبکه است و Autograd پیاده‌سازی عمومی آن در PyTorch است.
تصویربرداری، CNN و وظایف	CNN چرا برای تصویر مناسب است؟	دو فرض دارد: محلی بودن (پیکسل‌های نزدیک به هم مرتبط‌اند) و اشتراک وزن (یک فیلتر در همه جای تصویر استفاده می‌شود). این تعداد پارامتر را بسیار کم می‌کند.
تصویربرداری، CNN و وظایف	MaxPool چه می‌کند؟	در هر پنجره‌ی ۲×۲ بیشینه را نگه می‌دارد؛ ابعاد را نصف می‌کند و کمی مقاومت در برابر جابه‌جایی کوچک ایجاد می‌کند. ۲۸ ← ۱۴ ← ۷.
MASAI و غربالگری ماموگرافی	عدد دقیق MASAI چقدر است؟	عدد دقیق را از مقاله‌ی اصلی Lancet Oncology نقل نکرده‌ام و در فهرست راستی‌آزمایی گذاشته‌ام؛ ادعای کیفی این است که کشف سرطان افزایش و بار کاری کاهش یافته.
IDx-DR	خودمختار یعنی بدون پزشک؟	یعنی تفسیر تصویر توسط پزشک لازم نیست و خروجی مستقیم صادر می‌شود؛ اما در چارچوب کاربرد مشخص و مجوز همان کاربرد.
کشف دارو، AlphaFold و TxGemma	TxGemma دارو می‌سازد؟	نه. ویژگی‌های مولکول مثل سمیت را پیش‌بینی و ترکیبات را اولویت‌بندی می‌کند. آزمایش آزمایشگاهی و کارآزمایی بالینی همچنان لازم است.
ابزارهای واقعی و دستیارهای سلامت	چرا ChatGPT را ابزار تشخیص حساب نمی‌کنید؟	برای تحلیل تصویر پزشکی اعتبارسنجی و مجوز ندارد و ممکن است با اطمینان کامل پاسخ ساختگی بدهد (hallucination).
معیارهای ارزیابی	چرا F1 کلاس پنومونی ۰٫۸۹ است ولی Accuracy ۰٫۸۵؟	F1 فقط کلاس پنومونی را می‌سنجد که مدل در آن خوب است؛ Accuracy خطای زیاد کلاس نرمال (۸۶ مثبت کاذب) را هم شامل می‌شود.
معیارهای ارزیابی	Macro avg و Weighted avg فرقشان چیست؟	Macro میانگین ساده‌ی دو کلاس است (۰٫۸۲۵ برای F1)؛ Weighted بر اساس تعداد نمونه‌ی هر کلاس وزن می‌دهد (۰٫۸۴۱). در داده‌ی نامتوازن Macro صادق‌تر است.
پروژه‌ی من: مسئله و داده	چرا ۲۸×۲۸؟ این خیلی کم است.	PneumoniaMNIST استاندارد بنچمارک MedMNIST است؛ آموزش روی CPU سریع است و هدف یادگیری فرایند بود. محدودیتش را هم صریح در اسلاید محدودیت‌ها گفته‌ام.

بخش	پرسش	پاسخ کوتاه
پروژه‌ی من: مسئله و داده	داده از کجاست؟	MedMNIST v2، برگرفته از دیتاست رادیوگرافی قفسه سینه‌ی کودکان Kermany و همکاران؛ Train و Validation به نسبت ۹ به ۱ از Train اصلی و Test همان مجموعه‌ی جداگانه‌ی اصلی.
پروژه‌ی من: مدل و تصمیم‌ها	عدد ۱۵۶۸ از کجا آمده؟	۳۲ کانال × ۷ × ۷. دو MaxPool ابعاد ۲۸ را به ۱۴ و بعد ۷ می‌رسانند و padding=1 در کانولوشن ابعاد را ثابت نگه می‌دارد.
پروژه‌ی من: مدل و تصمیم‌ها	تعداد پارامترها را چطور حساب کردی؟	Conv1: ۱۶×۹+۱۶=۱۶۰؛ Conv2: ۳۲×۱۶×۹+۳۲=۴۶۴۰؛ fc1: ۱۵۶۸×۶۴+۶۴=۱۰۰۴۱۶؛ fc2: ۶۴×۲+۲=۱۳۰. جمع: ۱۰۵٬۳۴۶.
پروژه‌ی من: مدل و تصمیم‌ها	Loss وزن‌دار دقیقاً چه کرد؟	به خطای کلاس کم‌تعداد (نرمال) وزن بیشتری داد تا مدل فقط با گفتن «پنومونی» به دقت بالا نرسد.
پروژه‌ی من: انتخاب آستانه	چرا آستانه را روی Test تنظیم نکردی؟	چون آن‌وقت Test دیگر داده‌ی دیده‌نشده نبود و گزارش خوش‌بینانه می‌شد؛ Test فقط یک بار برای گزارش نهایی استفاده شد.
پروژه‌ی من: انتخاب آستانه	آستانه‌ی بهتر چیست؟	به هزینه‌ی خطاها بستگی دارد. با نتایج Test، آستانه‌ی بالاتر احتمالاً تعادل بهتری می‌دهد؛ روش اصولی انتخاب با معیاری مثل Youden روی Validation مستقل از بیمار است.
پروژه‌ی من: نتایج	چرا Specificity روی Test این‌قدر افت کرد؟	Validation از همان توزیع Train است ولی Test جداگانه جمع شده (Distribution Shift)؛ آستانه‌ی ۰٫۳ هم مدل را به سمت پنومونی متمایل کرده. نتیجه: ۸۶ مثبت کاذب از ۲۳۴ نرمال.
پروژه‌ی من: نتایج	ماتریس Validation در گزارش تو FN=23 دارد؛ چرا اینجا ۱۰ است؟	آن ماتریس با آستانه‌ی ۰٫۵ بود. برای مقایسه‌ی منصفانه هر دو را با ۰٫۳ آورده‌ام؛ عددهای ۰٫۳ از جدول جاروب آستانه آمده‌اند.
پروژه‌ی من: نتایج	با ResNet مقایسه کردی؟	از گزارش منتشرشده‌ی MedMNIST v2: ResNet-18 با ورودی ۲۸ روی همین Test به ACC=0.854 رسیده؛ مدل من 0.849 با حدود یک درصد پارامتر.
پروژه‌ی من: سیستم و دمو	پیش‌پردازش سرور با آموزش یکی است؟	سرور تصویر را خاکستری و ۲۸×۲۸ می‌کند و به بازه‌ی ۰ تا ۱ می‌برد. PneumoniaMNIST تصویر را اول از وسط مربعی برش داده؛ سرور فقط تغییر اندازه می‌دهد. این یک ناهماهنگی کوچک آموزش و سرویس (training–serving skew) است و در کار بعدی باید یکسان شود.
پروژه‌ی من: سیستم و دمو	۹۹٫۶۸٪ یعنی مدل درست می‌گوید؟	نه؛ یعنی مدل مطمئن است. احتمال‌ها کالیبره نشده‌اند و ۸۶ مثبت کاذب روی Test نشان می‌دهد اطمینان بالا لزوماً درستی نیست.
پروژه‌ی من: محدودیت‌ها	اگر وقت بیشتری داشتی اول چه می‌کردی؟	تقسیم داده بر اساس بیمار، ورودی با وضوح بالاتر و Transfer Learning، و بعد Grad-CAM برای اینکه ببینم مدل واقعاً به ریه نگاه می‌کند.
طراحی زبان: یک پروژه، چند زبان	Strong و Dynamic با هم تناقض ندارند؟	نه؛ دو محور مستقل‌اند. Dynamic یعنی نوع در زمان اجرا بررسی می‌شود؛ Strong یعنی تبدیل ضمنی ناامن انجام نمی‌شود. Python پویا و قوی است، JavaScript پویا و ضعیف‌تر (مثلاً "1" + 1 = "11").
طراحی زبان: یک پروژه، چند زبان	Structural typing یعنی چه؟	سازگاری بر اساس شکل نوع است نه نام آن؛ هر شیئی که فیلدهای Prediction را داشته باشد، Prediction حساب می‌شود. در مقابل Nominal typing مثل Java.
طراحی زبان: یک پروژه، چند زبان	Python مفسری است یا کامپایلری؟	هر دو: کد به bytecode کامپایل و روی ماشین مجازی CPython تفسیر می‌شود. TypeScript به JavaScript ترجمه می‌شود و موتور JS آن را با JIT اجرا می‌کند.

بخش	پرسش	پاسخ کوتاه
طراحی زبان: PyTorch به‌عنوان DSL	تفاوت DSL درون‌زبانی و مستقل چیست؟	مستقل مثل SQL نحو و پارسر خودش را دارد؛ درون‌زبانی مثل PyTorch از نحو و ابزارهای زبان میزبان استفاده می‌کند و یاد گرفتنش ساده‌تر است، ولی به محدودیت‌های میزبان هم مقید است.
طراحی زبان: PyTorch به‌عنوان DSL	Operator Overloading در PyTorch کجاست؟	عملگرهایی مثل + و * و @ روی Tensor با متدهای ویژه‌ای مثل __add__ و __matmul__ بازتعریف شده‌اند و هم محاسبه می‌کنند و هم در حالت گراد، گره‌ی گراف را ثبت می‌کنند.
طراحی زبان: PyTorch به‌عنوان DSL	Define-by-run چه عیبی دارد؟	چون گراف هر بار در زمان اجرا ساخته می‌شود، بهینه‌سازی کلی سخت‌تر است؛ برای همین PyTorch 2 ابزار torch.compile را اضافه کرد تا گراف را بگیرد و کامپایل کند.
طراحی زبان: سیستم نوع در مرز	Type erasure چه ضرری دارد؟	نوع‌ها در زمان اجرا نیستند، پس داده‌ی بیرونی (مثل JSON سرور) بررسی نمی‌شود. as فقط کامپایلر را ساکت می‌کند. راه‌حل: اعتبارسنجی زمان اجرا مثل Zod یا تولید نوع از OpenAPI.
طراحی زبان: سیستم نوع در مرز	Sound بودن سیستم نوع یعنی چه؟	یعنی اگر برنامه type-check شد، در زمان اجرا خطای نوع رخ نمی‌دهد. TypeScript عمداً unsound است (مثلاً به‌خاطر as و any) تا با کد JavaScript موجود سازگار بماند.
طراحی زبان: سیستم نوع در مرز	FastAPI چطور از type hint استفاده می‌کند وقتی Python آن را اجرا نمی‌کند؟	با Reflection: هنگام ثبت مسیر، امضای تابع و annotationها را می‌خواند (مثل typing.get_type_hints) و بر اساس آن ورودی را اعتبارسنجی و OpenAPI تولید می‌کند.
طراحی زبان: سیستم نوع در مرز	می‌شود شکل تانسور را در نوع آورد؟	تا حدی: PEP 646 (Variadic Generics) و کتابخانه‌هایی مثل jaxtyping اجازه می‌دهند ابعاد را در annotation بنویسیم و با ابزار بررسی کنیم؛ نسخه‌ی کامل آن به نوع‌های وابسته نیاز دارد.
طراحی زبان: ویژگی‌ها در کد	Decorator دقیقاً معادل چیست؟	@app.post("/predict") روی def predict معادل predict = app.post("/predict")(predict) است؛ app.post یک تابع برمی‌گرداند که تابع ما را ثبت و برمی‌گرداند.
طراحی زبان: ویژگی‌ها در کد	Context manager چه فرقی با try/finally دارد؟	از نظر معنایی همان است، اما الگو را در شیء کپسوله می‌کند (__enter__ و __exit__) تا تکرار نشود و فراموش نشود؛ شبیه RAII در C++.
طراحی زبان: ویژگی‌ها در کد	async در Python موازی است؟	نه؛ هم‌روند و همکارانه است. هر coroutine فقط در await نخ را رها می‌کند. کار CPUبر داخل async def حلقه‌ی رویداد را مسدود می‌کند؛ این ایراد کد فعلی است و باید به threadpool منتقل شود.
طراحی زبان: ویژگی‌ها در کد	چرا Lock لازم است؟	اگر درخواست‌ها در چند نخ اجرا شوند (مثلاً وقتی predict به def تبدیل شود و در threadpool برود)، قفل از دسترسی هم‌زمان به مدل جلوگیری می‌کند؛ در حالت فعلی async بیشتر جنبه‌ی احتیاط دارد.
طراحی زبان: درس‌ها	اگر دوباره می‌ساختی، زبان را عوض می‌کردی؟	برای مدل نه؛ اکوسیستم Python بی‌رقیب است. ولی قرارداد API را با Pydantic تعریف و نوع TypeScript را خودکار تولید می‌کردم تا مرز دو زبان امن شود.
سوگیری، حریم خصوصی و مقررات	اگر مدل اشتباه کند مسئولیت با کیست؟	پاسخ حقوقی در بیشتر کشورها روشن نیست؛ رویکرد رایج این است که پزشک تصمیم‌گیرنده‌ی نهایی بماند. ابزار من صریحاً برای تصمیم بالینی نیست.
آیا AI جای پزشک را می‌گیرد؟	بالاخره AI جای پزشک را می‌گیرد؟	شواهد فعلی نقش کمکی را نشان می‌دهد؛ حتی در پروژه‌ی کوچک من، مدل بدون نظارت انسانی یک‌سوم افراد سالم را اشتباه برچسب می‌زد.

بخش	پرسش	پاسخ کوتاه
جمع‌بندی	مهم‌ترین چیزی که از این پروژه یاد گرفتی؟	عدد Validation وعده‌ی دنیای واقعی نیست، و مرز بین زبان‌ها و نوع‌ها همان جایی است که باید بیشترین بررسی را داشته باشد.
عمومی	فرق Precision و Recall چیست؟	Recall (Sensitivity): از بیمارهای واقعی چندتا را پیدا کردیم. Precision: از کسانی که گفتیم بیمارند، چندتا واقعاً بیمار بودند. در Test: Recall پنومونی ۹۷٫۹٪، Precision ۸۱٫۶٪.
عمومی	مدل overfit شده؟	شکاف Validation و Test بیشتر نشانه‌ی Distribution Shift است، چون Validation از همان توزیع Train است. برای قضاوت درباره‌ی overfitting کلاسیک باید منحنی Loss آموزش و Validation را کنار هم دید.
عمومی	چرا Softmax با دو خروجی و نه Sigmoid با یکی؟	از نظر ریاضی معادل‌اند؛ دو خروجی با CrossEntropyLoss وزن کلاس را ساده می‌کند و به چندکلاسه تعمیم می‌یابد.
عمومی	AUC مدل چقدر است؟	در گزارش من نیست؛ قدم بعدی است. برای مقایسه، ResNet-18 روی همین Test در MedMNIST v2 به AUC=0.944 رسیده.
زبان	چرا TypeScript و نه JavaScript؟	سیستم نوع ایستا خطاهای شکل داده را قبل از اجرا می‌گیرد؛ مثلاً اگر prediction را با مقدار نامعتبر مقایسه کنم، کامپایلر خطا می‌دهد.
زبان	چرا FastAPI و نه Flask؟	FastAPI از type hint برای اعتبارسنجی و تولید OpenAPI استفاده می‌کند و async را بومی پشتیبانی می‌کند؛ ارتباط مستقیمی با مبحث سیستم نوع دارد.
زبان	GIL چیست؟	قفل سراسری مفسر در CPython که اجازه می‌دهد در هر لحظه فقط یک نخ bytecode اجرا کند. عملیات C++ در PyTorch آن را آزاد می‌کنند.
زبان	JIT و AOT چه فرقی دارند؟	AOT پیش از اجرا کامپایل می‌کند (مثل C++)؛ JIT در حین اجرا و بر اساس رفتار واقعی برنامه (مثل V8 یا torch.compile).
زبان	Duck typing یعنی چه؟	اگر شیء متد لازم را داشته باشد، نوعش مهم نیست. مثلاً transforms.Compose هر شیء قابل فراخوانی را می‌پذیرد.
زبان	Type Erasure فقط در TypeScript است؟	نه؛ Generics در Java هم با erasure پیاده‌سازی شده‌اند. در TypeScript همه‌ی نوع‌ها پاک می‌شوند.

۷. واژه‌نامه

اصطلاح	معادل فارسی	معنی کوتاه
Sensitivity / Recall	حساسیت	نسبت بیمارهای واقعی که پیدا شدند
Specificity	ویژگی	نسبت سالم‌های واقعی که درست رد شدند
Precision / PPV	دقت مثبت	نسبت پیش‌بینی‌های مثبت که درست بودند
Threshold	آستانه‌ی تصمیم	مرز تبدیل احتمال به برچسب
Distribution Shift	تغییر توزیع	تفاوت توزیع داده‌ی آموزش و داده‌ی واقعی
Data Leakage	نشت داده	ورود ناخواسته‌ی اطلاعات Test یا آینده به آموزش
Calibration	کالیبراسیون	هم‌خوانی احتمال پیش‌بینی‌شده با فراوانی واقعی
CNN	شبکه‌ی عصبی کانولوشنی	شبکه با فیلترهای محلی و وزن مشترک
DSL (embedded / external)	زبان تخصصی دامنه	درون‌زبانی یا مستقل
Define-by-run	تعریف حین اجرا	ساخت گراف محاسبه هم‌زمان با اجرا
Autograd	مشتق‌گیری خودکار	محاسبه‌ی خودکار گرادیان روی گراف
Operator Overloading	سربارگذاری عملگر	بازتعریف معنای عملگرها برای نوع جدید
Dynamic Dispatch	اعزام پویا	انتخاب متد در زمان اجرا بر اساس نوع واقعی شیء
Static / Dynamic Typing	نوع‌دهی ایستا / پویا	بررسی نوع در کامپایل یا در اجرا
Strong / Weak Typing	نوع‌دهی قوی / ضعیف	رد یا پذیرش تبدیل‌های ضمنی ناامن
Structural / Nominal	ساختاری / اسمی	سازگاری بر اساس شکل یا نام
Gradual Typing	نوع‌دهی تدریجی	ترکیب کد دارای نوع و بی‌نوع
Type Erasure	پاک‌سازی نوع	حذف اطلاعات نوع پیش از اجرا
Type Assertion	ادعای نوع	اعلام نوع بدون بررسی (as)
Type Narrowing	باریک‌سازی نوع	دقیق‌تر شدن نوع پس از شرط
Soundness	سلامت سیستم نوع	تضمین نبود خطای نوع در اجرا
Literal / Union Type	نوع لفظی / اجتماع	نوع با مقادیر مشخص / یکی از چند نوع
Reflection	بازتاب	خواندن ساختار برنامه در زمان اجرا
Decorator	آراینده	تابع مرتبه‌بالا که تابع دیگری را تغییر یا ثبت می‌کند
Context Manager	مدیر زمینه	مدیریت منبع با with
RAII	—	گرفتن منبع در سازنده و آزادسازی در مخرب (C++)
Coroutine	هم‌روال	تابع قابل توقف و ادامه (async)
Event Loop	حلقه‌ی رویداد	زمان‌بند هم‌روال‌ها روی یک نخ
Closure	بستار	تابع همراه با متغیرهای محیط تعریفش
JIT / AOT	کامپایل حین اجرا / پیش از اجرا	دو راهبرد کامپایل
GIL	قفل سراسری مفسر	محدودیت اجرای هم‌زمان bytecode در CPython

اصطلاح	معادل فارسی	معنی کوتاه
Dependent Types	نوع‌های وابسته	نوع‌هایی که به مقدار وابسته‌اند (مثلاً ابعاد)

۸. چک‌لیست روز ارائه و تمرین

روز قبل

- سه بار کامل با زمان‌سنج تمرین کن (کلید T در صفحه‌ی ارائه). هدف: ۱۹ تا ۲۰ دقیقه.
- بخش طراحی زبان (اسلایدهای ۱۸ تا ۲۲) را دو بار جداگانه تمرین کن؛ این بخش اصلی این درس است.
- بانک پرسش و پاسخ (بخش ۶) را مرور کن؛ به‌ویژه ماتریس Validation با آستانه‌ی ۰٫۵ و نقد async.
- پیش از ارائه ادعاهای فهرست راستی‌آزمایی را بررسی کن یا آن‌ها را محتاطانه بگو.

یک ساعت قبل

- در پوشه‌ی پروژه start.bat را اجرا کن؛ صبر کن پنجره‌ی بک‌اند پیام model loaded بدهد.
- در صفحه‌ی اصلی، وضعیت «سرویس مدل آماده است» را ببین و یک تصویر نمونه‌ی نرمال و یک پنومونی را تست کن.
- صفحه‌ی /presentation را باز کن، کلید F (تمام‌صفحه) و N (یادداشت) را امتحان کن.
- پشتیبان: اگر دمو کار نکرد، اسلاید ۱۶ عدد ۹۹٫۶۸٪ را دارد؛ بگو «نتیجه‌ی اجرای قبلی این است» و ادامه بده.
- اینترنت لازم نیست؛ فونت‌ها و همه‌چیز محلی است.

کلیدهای میانبر صفحه‌ی ارائه

کلید	کار
→ یا Space یا PageDown	اسلاید بعد
← یا PageUp	اسلاید قبل
N	نمایش یا پنهان کردن یادداشت گوینده و پرسش‌های احتمالی
G	نمای کلی همه‌ی اسلایدها
F	تمام‌صفحه
T	شروع یا توقف زمان‌سنج
Home / End	اولین / آخرین اسلاید
/presentation#18	پرش مستقیم به اسلاید ۱۸ (شروع بخش طراحی زبان)

۹. ادعاهای نیازمند راستی‌آزمایی

این موارد از متن اصلی تحقیق آمده‌اند. پیش از ارائه یا منبع اصلی را پیدا کن یا جمله را محتاطانه بگو.

ادعا	اقدام لازم
MASAI: افزایش کشف سرطان و کاهش بار کاری	مقاله‌ی اصلی، نوع پیامد، جمعیت، مقایسه و عدد دقیق از ناشر بررسی شود.
«دو برابر دقیق‌تر» در تصویربرداری سکته	منبع اصلی، تعریف دقت، نوع وظیفه، مقایسه‌گر و جمعیت مشخص شود؛ در غیر این صورت عدد حذف یا محتاطانه شود.
شناسایی حدود ۶۴٪ ضایعات در مطالعه‌ی صرع	مقاله‌ی اصلی و مخرج عدد بررسی شود؛ «ضایعاتی که متخصص ندیده» ادعای حساس و وابسته به طراحی مطالعه است.
IDx-DR و «اولین سیستم خودمختار مورد تأیید FDA در ۲۰۱۸»	صفحه‌ی رسمی FDA و اصطلاح دقیق تنظیم‌گری برای همان محصول/کاربرد بررسی شود.
«بیش از هزار ابزار AI مورد تأیید FDA»	تاریخ، تعریف فهرست و تمایز بین clearance/approval و تعداد دستگاه‌ها از صفحه‌ی رسمی FDA بررسی شود.
Flok Health و کاهش زمان انتظار تا نصف	منبع رسمی، جمعیت، دوره‌ی مقایسه و اینکه شاخص دقیقاً زمان انتظار است یا پیامد دیگری بررسی شود.
da Vinci: «برش کوچک‌تر، خونریزی کمتر، بهبودی سریع‌تر»	از تعمیم به همه‌ی جراحی‌ها پرهیز شود؛ سامانه‌ی رباتیک جراحی را خودمختار معرفی نکنید.
ChatGPT Health و دسترسی جغرافیایی	وضعیت فعلی محصول و دسترسی در زمان ارائه از صفحه‌ی رسمی OpenAI بررسی شود.
EU AI Act و دسته‌بندی هوش مصنوعی پزشکی	الزامات به کاربرد، طبقه‌بندی محصول و زمان‌بندی اجرایی وابسته‌اند؛ از عبارت مطلق «همه‌ی AI پزشکی پرخطر است» پرهیز شود.
پیش‌بینی سپسیس Epic در مطالعه‌ی ۲۰۲۱	مقاله‌ی JAMA Internal Medicine و جمعیت/نتایج دقیق بررسی شود؛ اعداد از خلاصه‌های ثانویه کپی نشوند.
AlphaFold و نوبل ۲۰۲۴	جایزه و نقش افراد/پژوهش‌ها از منبع رسمی Nobel Prize بررسی شود.
Halicin، abaucin و TxGemma	برای هر کدام مقاله یا مستندات اصلی جداگانه ذکر شود؛ شواهد محاسباتی/آزمایشگاهی از اثربخشی بالینی جدا بماند.

۱۰. منابع

پژوهش و شواهد بالینی

- Topol, E. (2019). Deep Medicine. Basic Books.
- Obermeyer et al., Science (2019) — algorithmic bias https://www.science.org/doi/10.1126/science.aax2342
- Epic Sepsis Model — external validation https://pubmed.ncbi.nlm.nih.gov/?term=External+Validation+of+a+Widely+Implemented+Proprietary+Sepsis+Prediction+Model
- MASAI mammography AI screening trial https://pubmed.ncbi.nlm.nih.gov/?term=MASAI+mammography+AI+screening+trial
- Halicin — deep learning antibiotic https://pubmed.ncbi.nlm.nih.gov/?term=halicin+deep+learning+antibiotic
- Abaucin — machine learning antibiotic https://pubmed.ncbi.nlm.nih.gov/?term=abaucin+machine+learning+antibiotic

کشف دارو و زیست‌شناسی محاسباتی

- Nobel Prize 2024 Chemistry https://www.nobelprize.org/prizes/chemistry/2024/summary/
- AlphaFold Protein Structure Database https://alphafold.ebi.ac.uk/
- TxGemma Model Card https://developers.google.com/health-ai-developer-foundations/txgemma/model-card
- Wang et al., TxGemma — arXiv (2025) https://arxiv.org/abs/2504.06196
- Google DeepMind — TxGemma https://deepmind.google/models/gemma/txgemma/
- Therapeutics Data Commons (TDC) https://tdcommons.ai/

مقررات و محصولات

- FDA — AI/ML-enabled medical devices https://www.fda.gov/medical-devices/software-medical-device-samd/artificial-intelligence-and-machine-learning-aiml-enabled-medical-devices
- HIPAA for Professionals https://www.hhs.gov/hipaa/for-professionals/index.html
- GDPR official text https://eur-lex.europa.eu/eli/reg/2016/679/oj
- EU AI Act https://artificialintelligenceact.eu/
- Lunit · Aidoc · Brainomix · Qure.ai · Ada Health
- OpenAI — Health in ChatGPT https://openai.com/index/health-in-chatgpt/

آموزش فنی

- PyTorch Tutorials https://docs.pytorch.org/tutorials/beginner/basics/intro.html
- PyTorch Autograd https://docs.pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html
- scikit-learn Model Evaluation https://scikit-learn.org/stable/modules/model_evaluation.html
- scikit-learn Probability Calibration https://scikit-learn.org/stable/modules/calibration.html
- scikit-learn Breast Cancer Dataset https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html
- MONAI Tutorials https://project-monai.github.io/tutorials.html
- Yang et al., MedMNIST v2 — Scientific Data (2023) https://arxiv.org/abs/2110.14795
- PneumoniaMNIST — TensorFlow Datasets https://www.tensorflow.org/datasets/catalog/pneumonia_mnist